User as Engram: Internalizing Per-User Memory as Local Parametric Edits

Bojie Li
Pine AI

Abstract

Personal memory in a language model is two problems, not one: *content* (the specific facts about a user) and *reasoning skill* (the ability to turn those facts into answers). The brain keeps the two apart (a sparse, local *engram* in the hippocampus for each episode, a slow neocortex for the shared skills that interpret it), so a new fact need not overwrite everything else. Most personalization today keeps a user's facts *outside* the weights, in a natural-language memory file or a retrieval index. When facts are written *into* the model instead, the standard recipe is the per-user LoRA adapter, which does the opposite of the brain, folding content and skill into one global weight delta. Writing a user's facts as a LoRA contaminates text unrelated to them; writing the same facts as local *Engram rows* leaves that text essentially untouched, with roughly 33,000× less extra loss.

We therefore propose **User as Engram**: store a user's content as surgical edits to the hash-keyed memory table of an Engram model, and carry the reasoning skill in one *shared* adapter. This layered design matches per-user LoRA's direct recall while delivering 5.6× higher indirect-reasoning accuracy on average, and never makes a single user worse at reasoning than the untouched base. The edit is a glass box: writing a fact switches on its lookup at exactly the trigger, adds the value the answer needs, leaves every other position unchanged to the last bit, and fails if written into the wrong layer. A follow-up study locates where the locality comes from. On the same facts, the Engram write leaves every layer before its read bit-identical and acts only at the trigger, while a LoRA changes every layer at every token. In Qwen3-8B, a single new fact stored in a gated neuron or a rank-1 LoRA leaks to unrelated people about half the time, but a subject-keyed edit (ROME) leaks far less: what keeps an edit local is a key, not a gate. Engram's hash address is such a key, though on the trigger's last tokens rather than on the subject. Training with counter-examples removes the leak from a LoRA too, but it costs two-hop reasoning in the middle and late layers. Because different users' facts land in disjoint hash slots, their edits *compose*: many users live in one shared table at once, stacking additively and losslessly, where a per-user LoRA, a single global weight delta, admits only one. Upon retrieval, a per-user Engram table does not grow with the population the retriever must search, so past ∼100 facts it overtakes a retrieval pipeline on a 2.5× larger model.

Code: https://github.com/19PINE-AI/user-as-engram
Website: https://01.me/research/user-as-engram

1 Introduction

Maya has talked to her AI assistant for the better part of a year. Over those months it has learned that her cardiologist is Dr. Elena Vasquez, that Vasquez practices at Lakeside Cardiology,

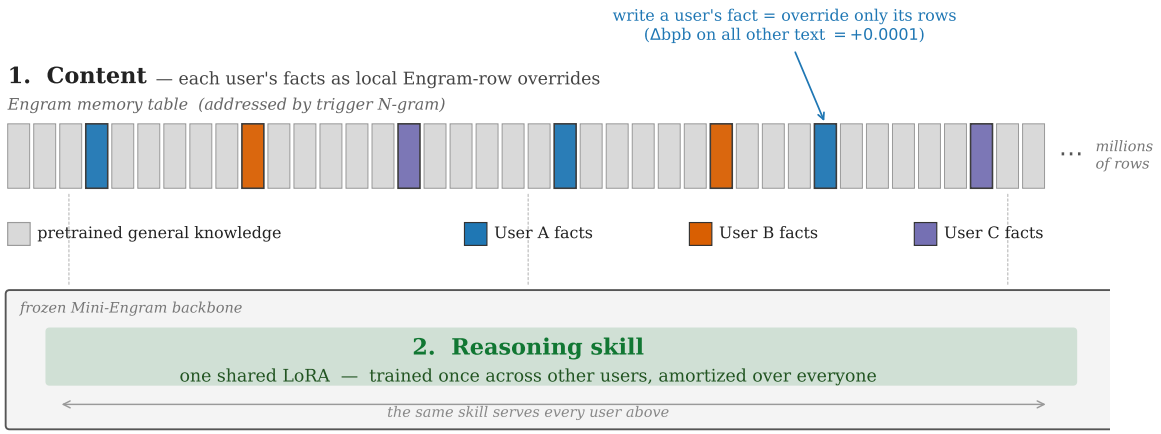


Figure 1. User as Engram splits personal memory into two layers, mirroring the brain's complementary learning systems. **Top (content).** Each user's facts are written as a few local row overrides (colored) in the Engram model's one hash-keyed memory table; the table's other rows hold the general knowledge learned at pretraining (gray). A write touches only that user's rows (Δbpb $= +0.0001$ on all other text), and different users hash to different addresses, so they never overlap: the fast, sparse, local trace of a hippocampal engram. **Bottom (reasoning skill).** A single shared LoRA on the frozen Mini-Engram backbone, trained once across other users and amortized over everyone: the slow, shared neocortex. The only per-user state is the handful of colored rows; everything below is shared.

that Maya is severely allergic to penicillin, and that she went vegetarian in 2024. A useful assistant must do three things with this *personal memory*: recall a fact on demand (*"who is my cardiologist?"*), reason over it when the question is indirect (*"I'm visiting my daughter in California next month, where should I go for a heart check-up?"*), and never let one user's facts leak into another's. Serving millions of such users from a single transformer [Vaswani et al., 2017] backbone is an open architectural problem.

Human memory already solves a version of it. The brain does not record a new fact by nudging every synapse a little; it writes the episode as a sparse, local trace (an *engram*) in the hippocampus, while the slow, distributed neocortex supplies the general skills that interpret it [Semon, 1921, Tonegawa et al., 2015, McClelland et al., 1995]. Keeping the fast, local store apart from the slow, shared one is exactly what lets us learn that Maya is allergic to penicillin without disturbing how we reason about allergies in general.

Personal memory in a language model has the same two-part structure, *content* (the user's specific facts) and *reasoning skill* (turning facts into answers), and the two pull in opposite directions. Content is private and differs from user to user, so it wants its own local store; the reasoning skill is common to everyone, so it should be learned once and shared (Figure 1).

Three families of methods address personal memory. In deployed systems the two that dominate are both non-parametric: in-context learning (ICL, Brown et al. 2020, Min et al. 2022), most often an automatically-extracted natural-language memory file, keeps the facts in the prompt, and retrieval-augmented generation (RAG, Lewis et al. 2020, Gao et al. 2023) fetches them at query time. Both leave the model's weights untouched, and we compare against retrieval head-to-head throughout (Sections 6.4–6.5). The third family writes the facts *into* the weights, and only this one conflates content and reasoning skill, storing both in the *same* parameters.

Among in-weights methods the standard recipe is a per-user LoRA [Hu et al., 2022] trained on each user's facts (Section 2). A LoRA has no address: to make one fact more likely, gradient descent bends the $Q/K/V/O$ directions that fit it most cheaply, and those same directions fire on unrelated text, so unless training explicitly pushes back, the edit is global. Pushing back is possible but not free: in a follow-up on Qwen3-8B, training each LoRA edit against counter-examples (other people, ordinary text) removes most of its leak, but costs two-hop reasoning once the edit sits past the earliest layers (Section 4.4). A global delta is also a single-tenant one: two users' LoRAs cannot share a model without merging into a combined delta that interferes, so each user needs their own copy of the weights.

Our approach: content in a memory table, skill in a shared adapter. **User as Engram** writes each of Maya's facts as a surgical edit to the hash-keyed memory table of an Engram-pretrained model [Cheng et al., 2026], whose gated lookups fire only on suffix N-grams via deterministic hashing; the reasoning skill is one *shared* LoRA trained once across a held-out population, so the only per-user state is the memory rows, swapped in per request. The table is *content-addressable* and inert off its key, so writing a fact changes the output only where its trigger N-gram is read (Section 4.3, Figure 7). That is why the table, not the weights, is the right home for a user's facts: writes are isolated, the suffix-N-gram index matches how personal facts are queried (by their surface form), and each row is a fixed $\sim$88 KB whose address swaps in per request, so storage grows with users, not model size. Isolation buys composability for free: because different users occupy disjoint addresses, their override maps commute and stack additively into one shared table, putting many users (or a corporate store layered under a personal one) in the same model at once (Figure 1, Section 5.3).

Organization of the paper. The rest of the paper develops four core claims:

1. **The cost of storing facts in shared weights** (Section 3). On a controlled base, writing a fact as a LoRA rather than as an Engram row is a $\sim$33,000$\times$ difference in extra loss on unrelated text (3-seed mean). This is a property of how a LoRA stores a fact when nothing in training tells it where *not* to act; in a single-fact follow-up, explicit counter-examples remove most of a LoRA's leak to other people, at a price in reasoning (Section 4.4). The extra loss is always there; the visible damage (worse reasoning) appears only on weak bases, and vanishes on a strong instruction-tuned base that answers the question on its own. We turn that damage on and off by changing base strength.

2. **User as Engram, with the mechanism opened up** (Section 4): a way to write a user's facts as small, local edits to an Engram model's memory table. Unlike a LoRA, whose change is spread invisibly across the whole model, every step of an Engram write can be watched on the trained model: it switches on the memory lookup at exactly its trigger, adds the value the answer needs and nothing else (every other position is unchanged to the last bit), and stops working if written into the wrong layer (Figures 7 and 8). A follow-up shows what that locality rests on: a key, not a gate. An added gated neuron is silent on most text yet still leaks a new fact to other people as often as a LoRA does, while subject-keyed edits leak far less (Section 4.4). We release four Mini-Engram checkpoints (178 M–1.22 B; a nano-scale public replication of Engram), all code and data, and a 50-line multi-tenant server with zero cross-user leakage.

3. **A layered design** (Section 6) that pairs per-user Engram content with one shared reasoning adapter (an artificial complementary learning system) and beats every all-in-one baseline on direct recall, indirect reasoning (5.6× over per-user LoRA on average), contamination, and per-user reasoning regressions. The lead survives a personal → medical schema shift.
4. **A deployment-scale comparison with retrieval** (Sections 6.4–6.5): which method wins is set by facts-per-user and population size, not benchmark size. Because a per-user Engram table does not grow with the population, it overtakes a strong retrieval pipeline (running on a 2.5× larger model) once the knowledge base passes ∼100 facts.

2 Background

The Engram architecture. Cheng et al. [2026], part of the DeepSeek line of sparse models [Dai et al., 2024, DeepSeek-AI, 2024], introduce *conditional memory* as a form of sparsity that complements mixture-of-experts (MoE, Shazeer et al. 2017, Dai et al. 2024). At each token position t, suffix N-grams of canonicalized tokens are hashed via K multiplicative-XOR heads into prime-sized embedding tables $E_{n,k}$. Retrieved rows e_t are projected by learned W_K, W_V and gated by an attention-style scalar $\alpha_t = \sigma(\mathrm{RMS}(h_t)^\top \mathrm{RMS}(W_K e_t)/\sqrt{d})$. The output $\alpha_t W_V e_t$ is added to the residual stream at the selected Engram layer. Crucially, the address $z_{t,n,k}$ is deterministic from the input token IDs—known before the forward pass—so the table can be offloaded to host DRAM without GPU contention.

LoRA-as-memory family. Per-user/per-document LoRA adapters [Su et al., 2025, Tan et al., 2024, Zhuang et al., 2024, Charakorn et al., 2025, Tan et al., 2024b, Bini et al., 2025] train a low-rank delta [Hu et al., 2022, Houlsby et al., 2019, Mangrulkar et al., 2022]—one member of a broad parameter-efficient fine-tuning family that also includes prefix-tuning [Li & Liang, 2021] and adaptive-rank and quantized variants [Zhang et al., 2023, Dettmers et al., 2023]—on $Q/K/V$ (and sometimes MLP) matrices on top of a frozen base. It is fit per user via NTP on a (observation, fact, QA) mixture, via knowledge distillation from a teacher (DyPRAG [Tan et al., 2025], DistilledPRAG [Chen et al., 2025]), or via hypernetwork emission (T2L [Charakorn et al., 2025]). All variants *edit the model weights globally*: every forward pass sees the LoRA's change.

Where User as Engram sits. These two—the Engram store and the per-user LoRA—anchor the landscape of personal-memory methods, which sorts along two axes (Figure 2): whether a method pays context tokens at query time, and whether storing a fact leaves the rest of the model untouched. In-context learning, retrieval, and external memory systems keep the model local but pay context on every query; per-user LoRA and knowledge editing pay no context but change shared weights. (How local such a weight edit is in effect depends on whether it is keyed: a subject-keyed editor such as ROME [Meng et al., 2022] leaks far less than a LoRA, Section 4.4, but it still rewrites a matrix every user shares.) User as Engram is the only one in the remaining corner—a local edit at zero context cost—and the rest of the paper earns that placement. We defer the broader literature to Section 9.

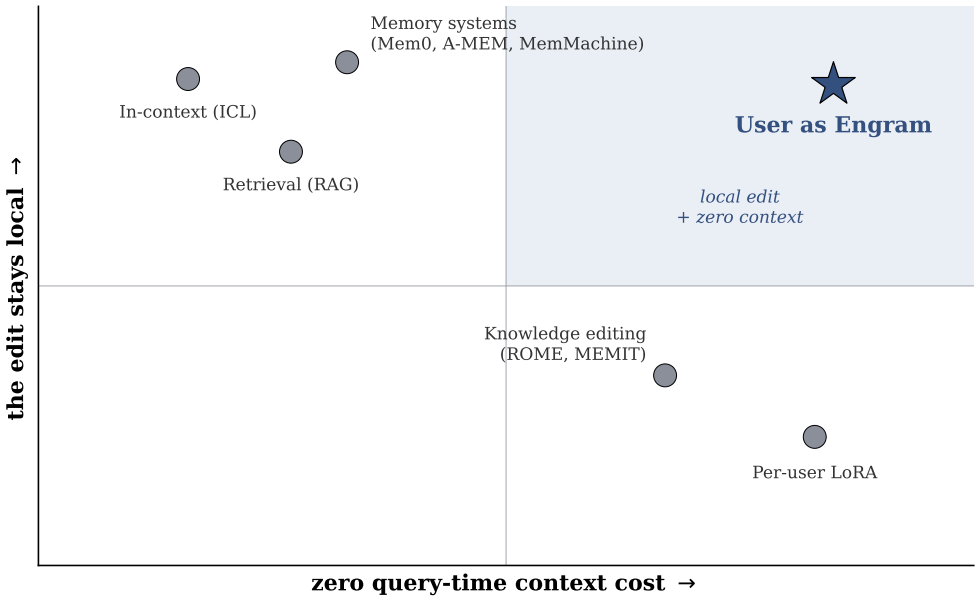


Figure 2. Where User as Engram sits among personal-memory methods. Context-based methods (in-context learning, retrieval, memory systems) leave the model untouched but pay context tokens at query time (top-left); weight-based methods (per-user LoRA, knowledge editing) pay no context but edit shared weights (bottom-right; a subject-keyed editor is closer to local in effect than a LoRA, Section 4.4). User as Engram occupies the remaining corner: a local edit at zero context cost. Axes are qualitative.

3 Per-user LoRA contaminates globally

Weights are an excellent place to keep knowledge—pretraining packs millions of facts into them accurately, at roughly 2–4 bits per parameter [Allen-Zhu & Li, 2025]. But that capacity is *shared* and the facts are *incompressible*: each must be stored explicitly and ends up entangled with every other—a distinct share of the weights that does not shrink with better training [Li, 2026b]. That is fine for the facts everyone queries. It is a poor place to keep *one user's* facts written as a *per-user weight edit*, and not for lack of capacity. A per-user LoRA edits $Q/K/V/O$ in every layer, so its edit is unconditioned (part of every forward pass) and must be fit per user from a handful of examples. Two problems follow, and we measure both: the edit is global, so it contaminates text that has nothing to do with the user; and the facts, once trained in, are *recalled but not readily reasoned over*. The first is an architectural cost paid on every base; the second matters most for personalization, and is why a fact belongs in a store the shared reasoning can read rather than in a per-user copy of the weights.

3.1 Architectural contamination on the same base (Mini-Engram-d20)

We hold the base fixed (Mini-Engram-d20@1536, 1.22 B parameters) and train a per-user LoRA (rank-64 on Q/K/V, 1,500 steps per fact, single-token gold) on the synthetic user fact sets used throughout. For each of 20 test users we measure val_bpb on a held-out 262 K-token ClimbMix shard—text unrelated to the user's facts—before and after the edit. Per-user Engram-row Joint OPT is measured under the identical protocol on the same base.

Figure 3 shows the gap; the mechanism is the point. A LoRA cannot write a fact without bending a function every input shares—to lift the probability of Maya's cardiologist it moves

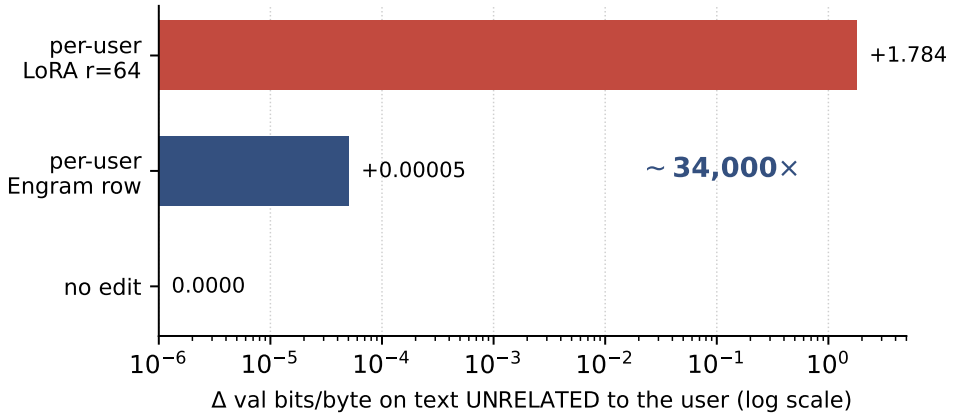


Figure 3. Architectural contamination on held-out text, Mini-Engram-d20 (canonical seed S0, n=20 users). A per-user LoRA more than triples val bpb on text *unrelated* to the user's facts (+1.784; per-user range [+0.44, +3.74]; 17/20 users worse), while a per-user Engram row leaves it unchanged to four decimals (+0.00005; 0/20 worse)—∼34,000× less, by design. The 3-seed mean ratio is ∼33,000× (Appendix O).

$Q/K/V/O$ along directions that also fire on unrelated text—so the loss on held-out text rises (more than *tripling* here), and it rises a little further with each fact added, because every new fact bends the function again. An Engram row is written to an address instead: it is read only when its trigger N-gram is hashed, and leaves the loss on everything else unchanged to four decimal places. This is the difference between editing a function and writing to a store, and it is precisely what makes co-locating many users feasible in one and hopeless in the other: contamination from a shared delta accumulates and cannot be quarantined per user, whereas addressed writes have zero cross-talk by construction (Section 7). The LoRA here is trained the standard way, on the user's facts alone. Section 4.4 shows that adding counter-examples to the training (other people, ordinary text, "answer as before") makes a single-fact LoRA edit far more local. That is a fix applied per edit and paid for in reasoning, not a property the LoRA has by itself.

3.2 Recall without reasoning

A per-user LoRA learns to *recall* its facts—direct recall is near-perfect—but the harder half of personal memory is *reasoning over* them, and that is where it struggles. The reason is composability. Knowledge acquired in pretraining is laid down in a form the model's reasoning can chain; a per-user adapter fit to a handful of examples instead memorizes a flat trigger→answer mapping that the shared reasoning does not readily pick up and combine. To answer an indirect question the adapter would have to carry the reasoning skill as well, learned from the same few examples—hard to train, and it does not transfer to facts it never saw. This recall-without-reasoning gap is the recurring finding of the per-user-adapter line [Su et al., 2025, Tan et al., 2024, Zhuang et al., 2024]: the facts go in, but indirect questions over them do not reliably come out.

Opening up per-user LoRAs on Qwen3-8B [Qwen Team, 2025b] shows what the flat mapping looks like (20 users, ∼650 facts, rank-64 LoRA on either $Q/K/V$ or the MLP; Appendix C.2). Direct recall goes from 0.01 to $\geq$ 0.997, but indirect accuracy only from 0.11 to 0.30

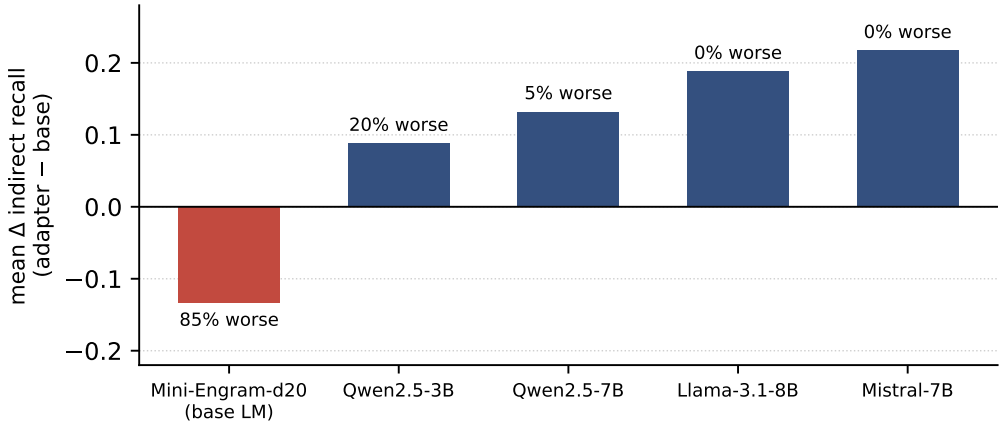


Figure 4. Cross-base LoRA scaling: mean change in indirect recall (adapter − base) with the fraction of users it hurts. On the base LM (Mini-Engram-d20) a per-user LoRA disrupts fragile completion behavior (85% worse, $\Delta{=}-0.133$); on every instruction-tuned base the reasoning skill absorbs the perturbation (0–20% worse, Δ from $+0.088$ to $+0.217$).

($Q/K/V$) or 0.35 (MLP). Even the $Q/K/V$ adapter reaches the answer logit mostly through MLP neurons (72% of its direct effect). Half of that neuron contribution sits in a median of 6–9 late-layer neurons, which the base already uses to write answer tokens [Geva et al., 2022]: the 100 neurons whose base weights best promote the answer carry 20–28% of it, against 0.02% for 100 random ones. Depth matters for reuse. Facts whose LoRA effect sits deeper than the median are answered indirectly far less often (0.06 vs. 0.58 for the MLP adapter). For the $Q/K/V$ adapter the gap is smaller (0.17 vs. 0.37), and the rank correlation is weak ($\rho = -0.06$).

What a user sees in addition depends on the base. We replicate the per-user LoRA recipe (rank-64 NTP on observation/fact/QA mixtures, 12 epochs $\times$ 200 samples) on four instruction-tuned bases—Qwen2.5-3B and Qwen2.5-7B [Qwen Team, 2025a], Llama-3.1-8B [Grattafiori et al., 2024], and Mistral-7B [Jiang et al., 2023]—and one base LM (Figure 4):

The split is clean (Figure 4). On a base LM the global perturbation also disrupts fragile completion behavior, so indirect recall drops outright; on an instruction-tuned base the base's own reasoning carries the indirect question and masks the adapter's inability to provide it. The strength of the base thus controls whether the damage is visible at all: we can make it appear or vanish by changing how strong the base is (Appendix P.9). The reasoning has to come from somewhere, and Section 6 supplies it from one shared skill instead of asking every per-user edit to learn it again.

4 Method: User as Engram

We instantiate the "local per-user edit" concretely: surgically write per-user fact rows into the hash-keyed memory table of an Engram-pretrained model. Figure 5 shows where that table sits in the network; the subsections below walk through how a fact is written into it (Figure 6) and how the writes are served per user (Figure 10).

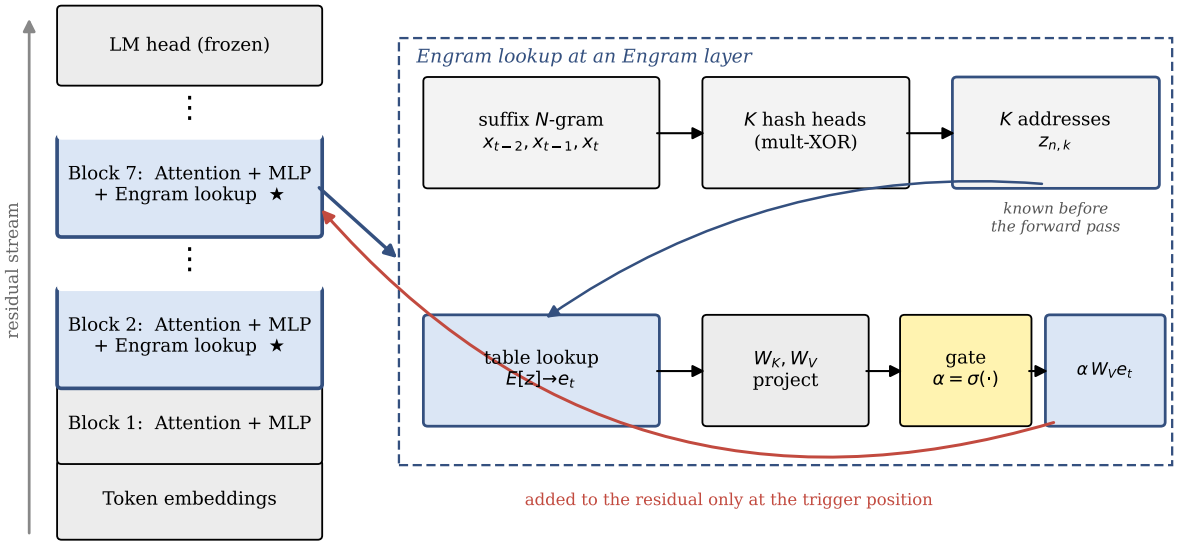


Figure 5. Where the Engram lives in the transformer. The backbone is a standard transformer; at a few designated Engram layers (★) a content-addressable lookup runs alongside attention and the MLP. At a position, the recent tokens' suffix N-gram is hashed by K heads into K table addresses (deterministic, hence known before the forward pass); the retrieved rows are projected by W_K, W_V, gated by α, and added to the residual stream—but the gate fires only where the trigger N-gram is present. Unlike attention and the MLP, which touch every position and every weight, the Engram read is sparse and addressable; that is what lets a single written row change behavior locally.

4.1 Where the Engram lives

Most of an Engram model is an ordinary transformer [Vaswani et al., 2017]; the only addition is a content-addressable lookup spliced into a few layers (Figure 5; two layers in our Mini-Engrams). At every position, the recent tokens—a suffix N-gram—are hashed by K deterministic multiplicative-XOR heads into addresses in a large embedding table E. The retrieved rows are projected by W_K, W_V, weighted by an attention-style gate $\alpha = \sigma(\cdot)$, and *added to the residual stream*, exactly where attention and the MLP also write. The lookup is thus a read from memory, but keyed by the surface form of the recent tokens rather than by query–key similarity across the sequence.

Two properties make this a genuine store rather than just more parameters, and both are what we exploit. First, the address is a deterministic function of the input token IDs—known *before* the forward pass—so we know exactly which rows a given query will read, and the table can be offloaded to host DRAM. Second, the gate makes the read sparse and conditional: most positions retrieve essentially nothing, and a stored row affects the output only at positions whose N-gram addresses it. This is the addressable, sparse store that per-user facts want—the opposite of the global weight edit of Section 3. Attention and the MLP are dense and global—every position, every weight—whereas the Engram read touches a handful of rows at a handful of positions, so editing one row changes the model only where that row is addressed. The rest of this section turns that into a per-user memory.

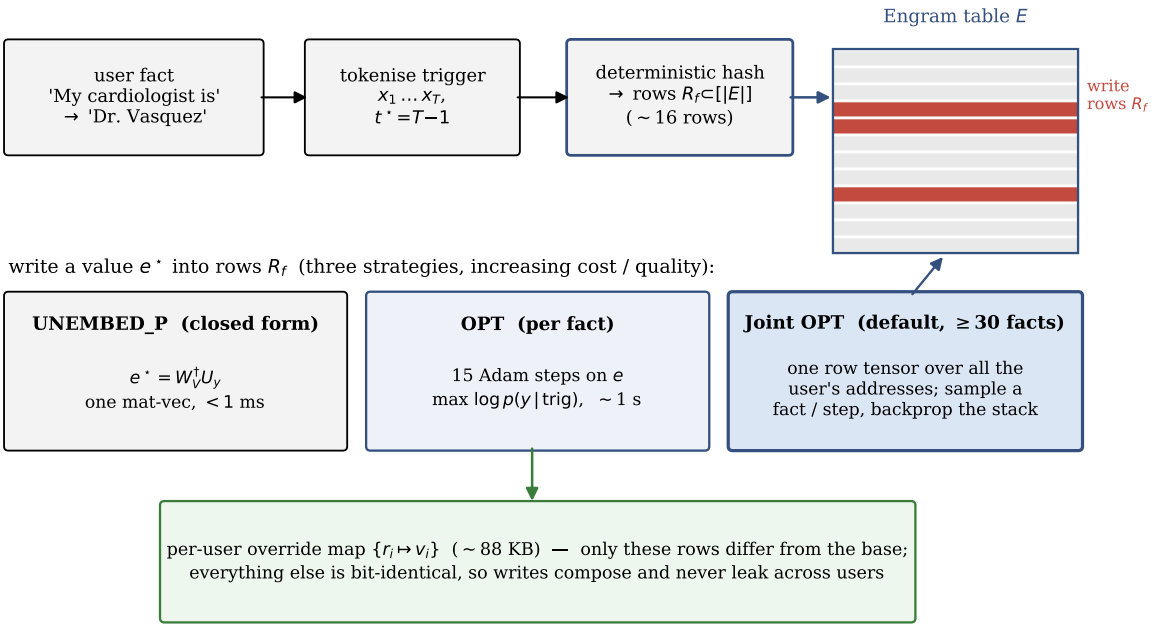


Figure 6. Inserting a user fact into the Engram. A fact is reduced to *where* and *what*: the trigger's suffix N-gram hashes to a sparse set of row addresses R_f (*where*), and one of three strategies writes a value $e^\star$ into those rows (*what*). UNEMBED_P solves for $e^\star$ in closed form; OPT takes a few gradient steps per fact; Joint OPT—our default beyond ∼30 facts/user— optimizes all of the user's rows together so they do not interfere. The result is a per-user override map of ∼88 KB; every row outside R_f, and the entire backbone, is left bit-identical.

4.2 Surgical row insertion

To write a fact, we put its answer where its trigger is read. The trigger's suffix N-gram hashes to a fixed, sparse set of table rows R_f—about 16 rows for our N=3, K=8 configuration—deterministically, touching no other row. What remains is *what* to write into those rows, for which we use three strategies of increasing cost and fidelity (per-strategy recall against a RANDOM control in Appendix E):

- **UNEMBED_P** (closed form): solve in one step for the row value whose projection steers the next-token distribution toward the answer—a single matrix–vector product, <1 ms, no training.
- **OPT**: refine that value with a few gradient steps per fact (∼1 s), trading time for accuracy.
- **Joint OPT** (our default beyond ∼30 facts/user): optimize *all* of a user's rows together rather than one fact at a time (full algorithm in Appendix N; convergence behavior in Appendix M).

Joint OPT is the default because of the one place this design has to earn its keep. When many of a user's facts are live at once, their rows are read in the same forward passes and interfere; optimizing them jointly lets the rows settle into values that coexist, which is what holds recall up as facts-per-user grows (Section 5). Crucially, every write still lands only in that user's R_f, so however many facts or domains are stacked, the store stays additive and private—the locality is the architecture's, not a constraint we have to impose.

Read together, the two figures give the method in one sentence: **remembering a fact is writing a few rows.** A fact decomposes into an address set R_f (*where*, fixed by the trigger's

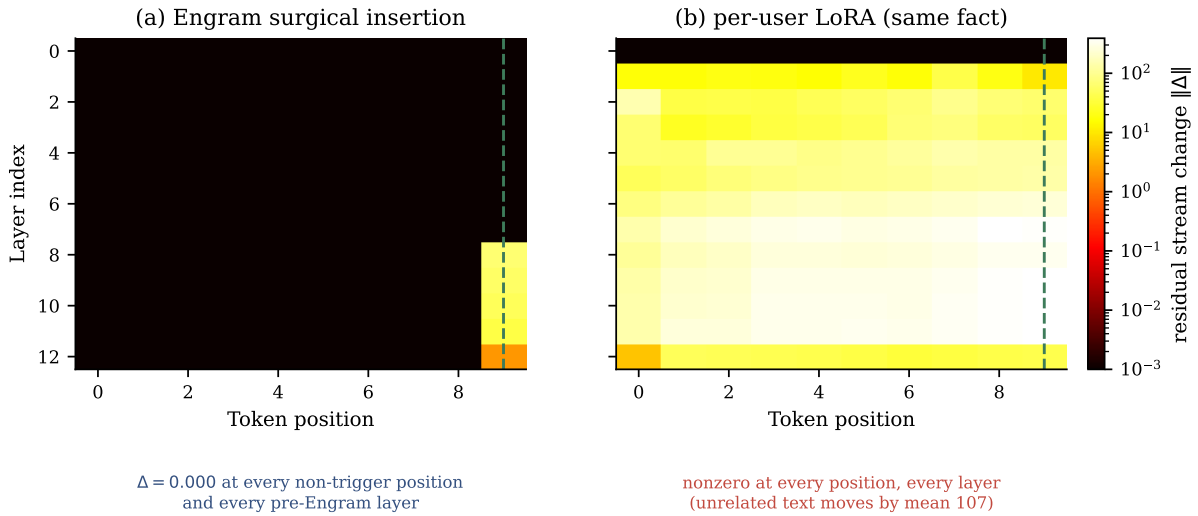


Figure 7. Addressed write vs. global function-bend. Per-layer, per-position residual-stream change $\|x^{(\ell)}_{\text{after}} - x^{(\ell)}_{\text{before}}\|$ on the *same* trigger sentence and base (Mini-Engram-d12@1280, log color scale). **(a)** An Engram row insertion is *exactly* 0.000 at every position before the Engram layer (causality) and at every non-trigger position after it—only the trigger column moves. **(b)** A per-user LoRA fit to the *same single fact* moves the residual at every position and every layer, and perturbs unrelated text ("The capital of France is") by mean 107. Storing a fact in a shared function changes the function everywhere; storing it at an address does not.

hash) and a value $e^\star$ (*what*, set by one of the strategies above), and the write touches nothing else. The locality we rely on is therefore *inherited from the architecture*—the read was already sparse and addressed (Section 4.1)—rather than enforced by a separate mechanism. That is why per-user and per-domain writes compose without a combiner and never leak across users (Section 5), and Section 4.3 verifies that "touches nothing else" is literally exact.

4.3 The local edit is a glass box

A LoRA's change is spread invisibly across the whole model, so you cannot point to what it did. An Engram write is the opposite: you can watch every step of it on the trained model. We follow one inserted fact through the write and check each step on the trained Mini-Engrams (full detail, with the depth and multi-hop probes, in Appendix B). Three things are worth seeing, and Figures 7 and 8 show each.

(1) The write switches on its own lookup, and adds just the value it should. A row reaches the output only through a gated value, $\alpha_t W_V e$: a switch α_t that decides whether the lookup is read, times the value $W_V e$ the row carries. Writing a fact does both at once. The switch at the trigger turns from nearly off to nearly on, while it stays off everywhere else (Figure 8a)—the row we write is also what turns its own lookup on. And the change the row makes to the output points almost exactly along the value it carries (Figure 8b): the switch and the small convolution around it scale the value but do not turn it into something else. The same alignment was reported for the original Engram code on a fresh, untrained model; we confirm it on the trained model, for the strategy we actually deploy.

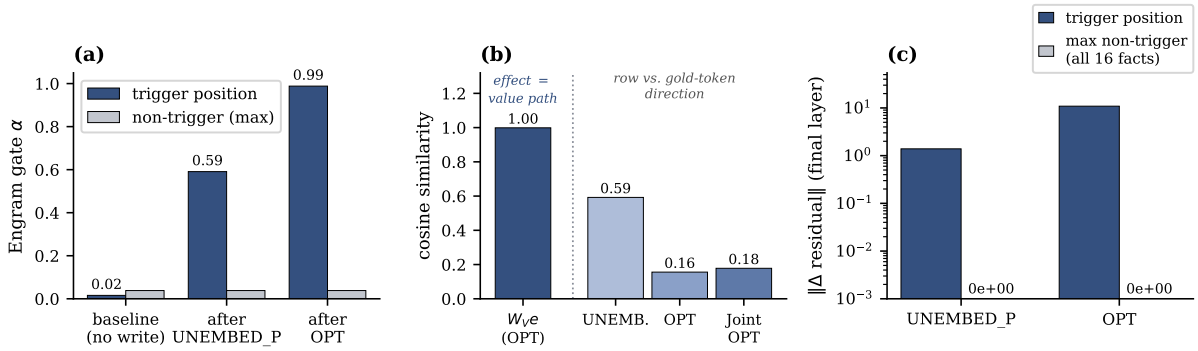


Figure 8. The mechanism on the trained model (Mini-Engram-d20, 16 facts). **(a)** The write opens its own gate: the trigger-position gate α rises from 0.02 to 0.99 (OPT) while non-trigger positions stay at 0.04. **(b)** The deployed row's residual change is cosine 0.999 to its value-path projection $W_V e$ (left bar); what the row *encodes* relative to the gold token's unembedding is exact for the closed-form solution (0.59) and drifts as gradient OPT/Joint-OPT trade direct gold-alignment for higher recall (0.16–0.18). **(c)** Exact locality: the maximum non-trigger residual change over all 16 facts is 0.000 for both strategies, versus an order-1 change at the trigger.

(2) Nothing else moves—exactly nothing. We write only the trigger's rows, and the trigger's last few tokens are unique in the sentence, so every other position reads the rows it always read and its output is unchanged to the last bit. This holds for every one of the 16 test facts and both writing strategies: the largest change at any non-trigger position, at any layer, is 0 (Figure 7a). The same holds when a whole user is written at once with Joint OPT: over all 20 test users (474 facts) the residual entering every block up to the Engram read is unchanged, and non-trigger positions are bit-identical for 473 of the 474 facts (Section 4.4). A LoRA that learns the same fact does the opposite—it moves every position in every layer, and shifts unrelated text as well (Figure 7b). This is the line between editing a function and writing to a store. Note where the locality comes from: not from the switch being off elsewhere, but from the fact that we only ever changed the rows at one address.

(3) It only works near the end of the network. The Engram lookup is read at a late layer, after the model has mostly made up its mind about the next token. That is why a row can change the answer cleanly. To check this is the real cause and not a coincidence, we write the same facts into the model's *early* lookup instead: at the same effort, recall drops from perfect to about a quarter, because an edit made early gets reworked by all the layers above it. Where the lookup sits is not a free choice—it has to be late enough that the value it adds is close to the final answer.

What this kind of write *cannot* do is join two facts together. The lookup only matches the words it was given, so it answers "who is my doctor" but not "where does my doctor work" unless that second step was stored too (Section 5.6). That is the reason we keep the facts and the reasoning apart (Section 6): the store can hold a fact but cannot chain it, so the chaining has to come from somewhere else. For this store, the split between facts and reasoning follows from how the mechanism works; it is not a convenience we chose. Locality alone does not force it. In Qwen3-8B, a subject-keyed ROME edit in an early layer has a leak to other people of only 0.3% (defined in Section 4.4) and still supports a second hop through pretrained knowledge (0.49, vs. 0.52 with the fact in context) (Section 4.4). What rules out chaining here is the surface-N-gram

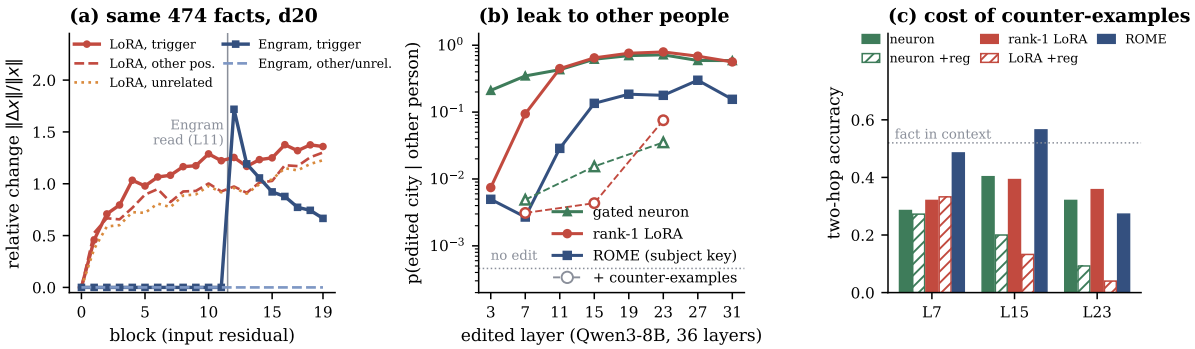


Figure 9. Locality comes from a key, not a gate. (a) Mini-Engram-d20, all 20 test users' facts (474 facts the base did not already know), written as Engram rows (Joint OPT) or as the standard per-user LoRA. Both recall all 474. Mean relative residual change per block: the Engram write is exactly zero before its read at layer 11 and at every non-trigger position and unrelated sentence (473 of 474 facts bit-identical off-trigger; the remaining one moves by at most 2%). The LoRA moves every block, at the trigger, at the prompt's other positions and on unrelated text alike. **(b)** Qwen3-8B, one fictional fact ("X lives in the city of Y") per edit, 200 facts per point: mean probability the edited model gives Y when asked where 40 *other* people live. A new gated MLP neuron leaks as much as a rank-1 LoRA; ROME, keyed on the subject, leaks least. Hollow markers: the same edits trained with counter-examples. **(c)** Two-hop accuracy (asking for Y's country) at the counter-example layers: specificity is free at L7 but halves two-hop or worse at L15 and L23.

key and the late read.

4.4 Where the locality comes from: a key, not a gate

The glass box shows *that* an Engram write is local. A follow-up study (Appendix C) asks *why*, from two sides: the same facts written as Engram rows and as a LoRA, and single facts written into a large dense model by edits that do or do not have a gate or a key (Figure 9).

Same facts, different edits. On Mini-Engram-d20 we wrote each of the 20 headline test users' facts both ways: Engram rows by Joint OPT, and the standard per-user LoRA (rank-64 on $Q/K/V$, 1,500 steps, as in Section 3). We kept the 474 facts the base did not already answer. Both methods recall every one of them, so any difference lies in how they store a fact, not in whether they do. The Engram write leaves the residual stream entering blocks 0–11 exactly unchanged and acts only at the trigger; the LoRA already moves the residual by 46% of its norm after the first block, at the trigger, by 53% at the prompt's other positions, and by 37% on unrelated sentences, growing to 120–140% by the last block (Figure 9a). Downstream, the two edits use partly different neurons. Ranking MLP neurons by their attributed effect on the answer logit (activation $\times$ gradient), the two methods' top-100 sets have a median Jaccard index of 0.24, so about 39 of 100 are shared. Resetting a method's own top-100 neurons to their base values leaves 0.42 (Engram) or 0.40 (LoRA) of its effect; resetting the *other* method's top-100 leaves about 0.60; 100 random neurons leave all of it. The Engram row also works through the MLPs that read it (0.82 of its effect attributed to MLP neurons, against 0.25 for the LoRA). So an Engram row is not a localized copy of what a LoRA does; it is a different, addressed edit that feeds part of the same answer-writing machinery.

A gate is not a key. An MLP neuron reads one direction and writes one direction, the shape of a rank-1 LoRA, plus a gate that can switch it off. If locality came from gating, a new neuron should store a fact as locally as an Engram row. We tested this in Qwen3-8B [Qwen Team, 2025b]: 200 fictional people, one "lives in" fact per edit, written at one of nine layers (3–35) as a new gated SwiGLU neuron, a rank-1 LoRA on the MLP's down or up projection, or a ROME edit [Meng et al., 2022], about 9,200 edits in all (Appendix C.3). Every method recalls the fact. We call the mean probability an edit gives the new city when asked where 40 *other* people live its *leak* (no edit: 0.05%). The gate does make the neuron sparse: it fires on only 0.5–5% of ordinary text tokens at layers 7–31, while a rank-1 LoRA's change fires on up to 94%. But it does not make the neuron specific to the subject. Averaged over layers 3–31, the neuron's leak is 53%, the same as the rank-1 LoRA's (44–50%) (Figure 9b). Left to itself, the gate learns to open on "someone's home city is being asked", not on this person. ROME, which keys its edit on the subject's representation, leaks 12% on average and under 3% in layers 3–11 where it is normally applied. This is the stage-(3) observation of Appendix B, that Engram's locality comes from addressing and not from its gate, now seen from the other side: in a dense model, too, locality comes from a key on the input.

Counter-examples also work, at a price. A gate *can* learn to be specific if it is shown where not to open. We repeated the edits at layers 7, 15 and 23, adding a KL-to-base term on other people with the same relation, other relations, and generic text, and gave the LoRAs the same counter-examples. The leak fell from 35%/62%/72% to 0.5%/1.5%/3.5% for the neuron, and from 9–79% to 0.3–8.5% for the two rank-1 LoRAs (a rank-16 $Q/K/V$ LoRA on every layer: 50% to 1.6%). So the gate is not what makes an edit subject-specific, and a LoRA can be made as local to its subject as a neuron. This weakens any claim that a LoRA is non-local *by construction*: it is non-local when trained, as per-user LoRAs normally are, on the user's facts alone. The counter-examples have three costs. They are needed for every edit. They do not make the LoRA sparse on ordinary text: past the earliest layers the regularized rank-1 LoRA still moves the residual on most text tokens (67% at layer 15, 94% at layer 23, against 20–36% for the neuron), and at layer 23 its unit fires on about half of them (the neuron's on 7%). And they cost composition: at layers 15 and 23, two-hop accuracy halves or worse (neuron 0.40 $\rightarrow$ 0.20 and 0.32 $\rightarrow$ 0.09; rank-1 LoRA 0.39 $\rightarrow$ 0.13 and 0.36 $\rightarrow$ 0.04), while at layer 7 it is unchanged (Figure 9c). ROME at layer 7 remains the best single edit: 0.3% leak, 0.59 on reworded questions, 0.49 two-hop, close to giving the fact in context (0.66 and 0.52).

What this means for User as Engram. An Engram row gets its locality the way ROME does, from a key, but with no training against counter-examples and no change to the shared weights. Two qualifications follow from the data. First, Engram's key is the trigger's last two or three tokens, not the subject. Two people's facts are kept apart by the per-user override tables, which make the user the implicit subject, and by the subject's name only when it falls inside the trigger N-gram. We did not measure leakage between subjects that share a relation phrase inside one table. Second, the depth sweep explains part of the composition gap. Every method's two-hop accuracy collapses when the fact is written into the last quarter of the network (0.02–0.05 at layer 27 and 0.00 at layers 31–35, with direct recall still ≈ 1.0), whereas edits at layers 7–19 compose (0.26–0.61), consistent with multi-hop failures after knowledge editing [Zhong et al., 2023]. Engram's read sits after 55–60% of the network, so its inability to

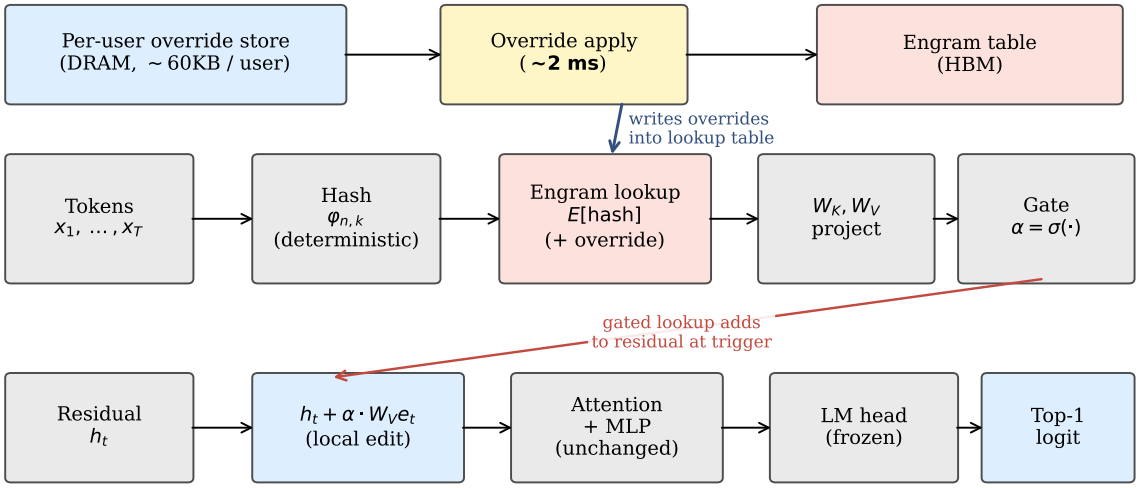


Figure 10. EngramServer architecture. Per user, an override map of (row index, row vector) pairs lives in DRAM. On each request, the server saves the originals at the affected addresses (∼2 ms), writes the user's overrides, runs the forward pass, then restores. The Engram lookup at the configured layer transparently sees the override values; the gate fires only at the trigger N-gram. There is no router and no graph rewrite.

chain (Section 5.6) comes mainly from its surface key. Finally, adding Engram to pretraining leaves the base model's own neurons as interpretable as a dense model's (Appendix C.4).

4.5 Per-user override tables and additive composition

The production design uses per-user override tables. Each user has a small dictionary $\{r_i \mapsto v_i\}$ of fact rows. At request time, the server saves the originals at the affected addresses, writes the user's overrides, runs the forward, and restores. Cross-user leakage is *zero by design*—user A's overrides are not in the table when user B queries. (We also benchmarked a shared table with a per-user hash salt and rejected it for lower recall; Appendix J.)

Override maps with disjoint addresses commute, so corporate facts + user facts (or any number of domain-specific Engrams) **stack additively at inference** without retraining a combiner. This mirrors how Stable Diffusion LoRAs additively stack, but at the row level. Section 5 confirms that additive composition is lossless when the domains have disjoint trigger templates.

5 Experiments

Before composing the store with a shared reasoning skill (Section 6), we first characterize the bare per-user store on its own—how well facts surface, what they cost, and where it breaks—so that the layered design's gains can be attributed cleanly to the skill it adds. This section answers four questions about User-as-Engram, in order. **(a) How accurately do facts surface?** (Section 5.2) for one fact and for many at once. **(b) What does it cost?** (Section 5.3) storage and composition vs. a per-user LoRA. **(c) Does writing the fact beat retrieving it?** (Sections 5.4–5.5) against memory-system and RAG baselines on the same answering LM, including paraphrased

Table 1. Mini-Engram pretraining at the ablation-optimal recipe. All use the `large` Engram (50 K × 256 = 51.2 M Engram params). Trained on a single NVIDIA RTX PRO 6000 Blackwell (102 GB) in bf16.

	d8 v2	d12 v2	**d12@1280 opt.**	**d20@1536 opt.**
depth × width	8 × 512	12 × 768	12 × 1280	20 × 1536
Engram layers	2, 5	2, 7	2, 7	2, 11
total params	178 M	339 M	625 M	**1.22 B**
scaling-params	42 M	110 M	278 M	617 M
ClimbMix tokens (12 t/p)	0.50 B	1.32 B	3.34 B	7.40 B
final val_bpb	0.912	0.827	0.770	**0.730**
wall-clock (single GPU)	∼50 min	∼5 h	∼10.5 h	∼70 h (shared GPU)

queries. **(d) Where does it break?** (Section 5.6) multi-hop chaining—the store matches words but does not compose them. The capacity, token-budget, and dense-size scaling ablations that justify our recipe are in Appendix P. We use three fact corpora throughout: the **base** 200-fact benchmark (100 USER + 100 ORG facts), the **XL** corpus of 1,000 USER + 1,000 ORG templated facts (Section P.3), and the **XXL** corpus of 3,132 distinct trigger templates (high-density and distinct-template stress tests).

5.1 Setup: the Mini-Engram models we insert into

Before measuring per-user insertion we need an Engram backbone to insert into. Cheng et al. [2026] released only architectural reference code; no Engram-trained weights are public. We graft Engram into Karpathy's nanochat [Karpathy, 2026]—a GPT-2-style backbone [Radford et al., 2019] optimized with Muon+AdamW [Jordan et al., 2024]—and train four Mini-Engram models on a single Blackwell GPU at the ablation-optimal recipe (Section P.2; pretraining curves in Appendix D): the same `large` Engram table (50 K × 256, 51.2 M params), trained at Karpathy's 12 tokens-per-param (t/p) budget on the scaling-params for every dense size.

Model-name notation. Throughout the paper we use `d{depth}@{width}` for the ablation-optimal Mini-Engrams (`d12@768`, `d12@1280`, `d20@1536`) and `v1`/`v2` to denote earlier and final training runs at the same shape. Where the width is omitted (e.g. `d12` without a suffix), we mean the `v2` (ablation-optimal at width 768, 339 M total) checkpoint. The four headline checkpoints, their param counts and token budgets are in Table 1; all four share the same 51.2 M Engram table.

We reproduce Engram's two signature findings. Suppressing the Engram lookup hurts factual recall far more than reading comprehension—the factual-vs-reading sensitivity asymmetry of Cheng et al. [2026] (their §6.3), at the ∼10× smaller magnitude expected for a model two orders smaller on a corpus three orders smaller. The Engram layers also act as effective extra depth: a LogitLens probe shows our small Engram model resolving its prediction earlier than its dense-only twin, matching their §6.1 (Appendix Figure 22).

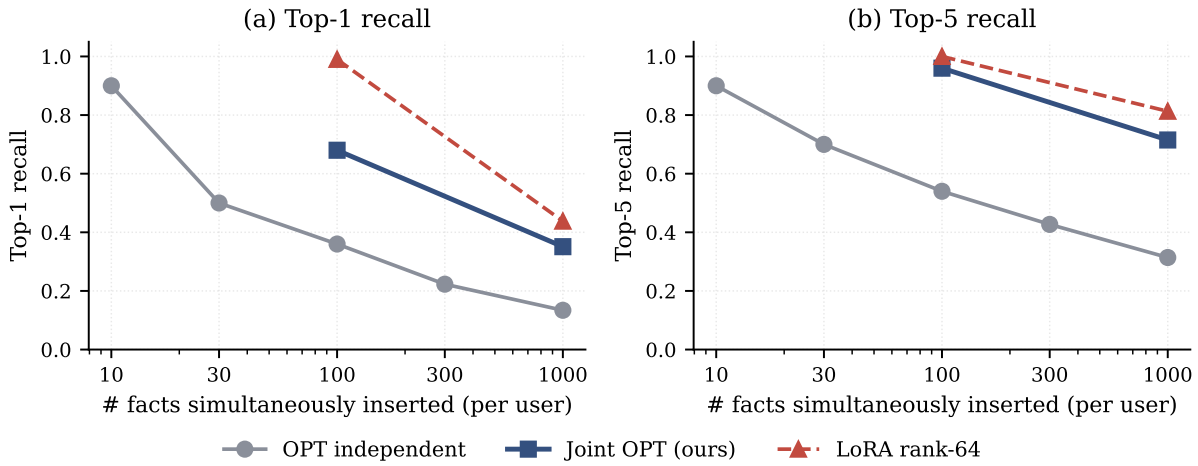


Figure 11. Within-user fact-density: top-1 (left) and top-5 (right) recall vs. number of facts inserted simultaneously into one user's override map. Joint OPT (blue) closes most of the gap to LoRA rank-64 (red dashed) at 161× less storage (88 KB vs. 14.2 MB at 100 facts).

5.2 Per-fact recall and within-user density

A single inserted fact surfaces within ∼8 points of the in-context ceiling, at three orders of magnitude lower per-user cost than training a full LoRA (Appendix E). The harder question is what happens when many facts are live at once: the rows in a single user's override table interfere during the forward pass. We sweep that density curve on Mini-Engram-d12 (Figure 11).

Joint OPT is the recommended default for ≥30 facts/user: it tracks multi-fact LoRA rank-64 to within a few top-5 points while keeping top-5 recall above 90% out to ∼300 facts/user, all at up to ∼161× less per-user storage (Figure 11; per-n recall and storage in Appendix P).

What the density ceiling is—and what it is not. Three ablations triangulate the ceiling to a single cause, ruling out the obvious suspects. (i) It is *not* backbone capacity: holding Engram capacity fixed and scaling the dense backbone does not lift recall at 1 000 facts—it slightly degrades it (top-1 0.35 → 0.28 from d12@768 to d20@1536, Section 8.3). (ii) It is *not* the table or the hash: per-fact *independent* OPT, where each fact gets private rows, shows *no* ceiling to 1 000 facts (≥ 0.98 top-1, Appendix P.3). (iii) It is *not* pretraining: a multi-fact-in-the-loss finetune only accelerates convergence—given enough OPT steps the baseline reaches the same asymptote (Section P.1). What remains is the one thing all three share: **gradient interference among the co-active rows in a single user's table, at a fixed inference-time optimization budget**. This is why the path to breaking it is a recipe change (multi-fact-in-the-loss, scoped tables), not further dense scaling, and why it scales with facts-per-user rather than with the model.

5.3 Cost, storage, and composition

Storage. A user's whole table is ∼88 KB at 100 facts/user and grows about linearly with the fact count (∼0.9 KB/fact), while a per-user LoRA is a fact-count-independent 14.2 MB—∼161× larger at 100 facts/user, and ∼1700× at 10 facts/user. At a million users that is 100 GB versus 14.2 TB: the difference between one server and a distributed store (Appendix Figure 38).

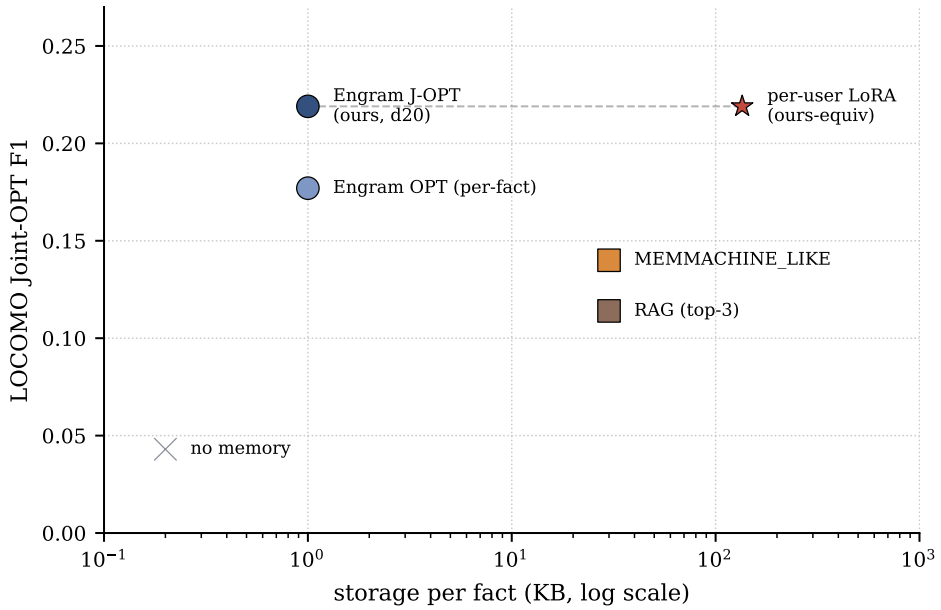


Figure 12. Cost-quality trade-off across personal-memory methods at the same answering LM (Mini-Engram-d20@1536). User-as-Engram Joint OPT matches a per-user LoRA's LOCOMO F1 at 161× smaller per-user storage (88 KB vs. 14.2 MB at 100 facts). Retrieval baselines store each fact at a higher per-fact cost than an Engram row and still cap out below Engram on quality.

Figure 12 places all the methods on one cost–quality plot: a single fact matches the in-context ceiling at a fraction of LoRA's cost, and at high density Joint OPT keeps pace with even rank-64 LoRA at tens of times less storage, with the gap turning in Engram's favor by 1 000 facts.

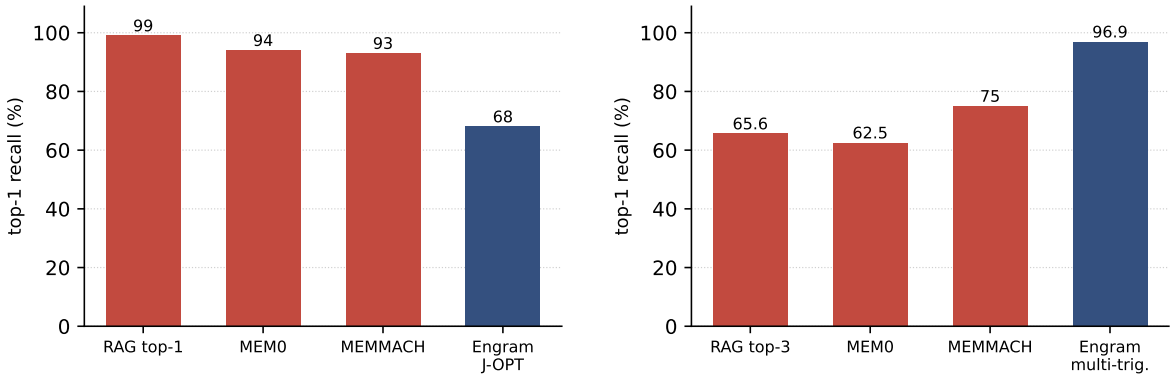
Composition. Two users' tables, or a user's facts and a company's, occupy different addresses, so they simply add up—no combiner to train. The one exception is when two sets of facts share the same trigger words: their addresses collide and the last write wins. The rule, then, is to add tables when their triggers differ and to keep separate per-user tables when they do not (Appendix G).

5.4 Comparison against memory systems

We compare User-as-Engram against state-of-the-art memory systems that target the same use case (per-user / personal memory). All systems use the *same* Mini-Engram-d12 as the final-answer LM; the only difference is how facts are stored and retrieved. Our `MEM0` and `MEMMACHINE` baselines follow the retrieval recipes of Mem0 [Chhikara et al., 2025] and MemMachine [Wang et al., 2026]. The sentence encoder used by the RAG / MEM0 / MEMMACHINE baselines is `all-MiniLM-L6-v2` [Wang et al., 2020, Reimers & Gurevych, 2019] (80 M parameters; comparable in size to a production retriever).

This subsection probes *direct* fact recall under trigger-exact and paraphrased queries; the separate *indirect-reasoning* comparison (does retrieval recover the gold fact across hops, and what happens as the KB grows?) is deferred to Sections 6.4–6.5.

Setup. 100 USER facts (XXL corpus), asked two ways: with the fact's exact stored prompt (easy for retrieval—the words match), and with a paraphrase ("I love the spice" for "My favorite spice is"; 16 facts × 4 rewordings).



(a) Retrieval wins on exact-trigger queries—but Engram pays *zero* context tokens (0 vs. 16–63) for its 68%.

(b) Engram multi-trigger insertion wins on paraphrases by 22 points, again at zero context.

Figure 13. Direct recall vs. memory systems, all sharing the same Mini-Engram-d12 answering LM (100 USER facts, XXL corpus). a When the query is the fact's stored prefix, nearest-neighbor retrieval is near-perfect and beats Engram's 68% top-1—but at a context-token cost Engram does not pay. b When the query is a paraphrase, retrieval drops to 60–75% while multi-trigger Engram insertion reaches 96.9% top-1.

On exact-trigger queries the words line up with what was stored, so nearest-neighbor retrieval is near-perfect and beats Engram's 68% top-1—but it pays 16–63 context tokens for the answer, where Engram pays none (Figure 13).

The story flips on paraphrased queries. When the query is reworded, the sentence encoder confuses surface forms and retrieval slips, while a single-trigger Engram misses the reworded N-gram entirely. The fix is to write a row at every anticipated paraphrase: multi-trigger insertion then wins on paraphrase by a clear margin at zero context cost (Figure 13), because the contract is explicit—the trigger N-gram—rather than left to an encoder's notion of similarity.

Scale invariance. This verdict (*retrieval wins on the verbatim prefix; multi-trigger Engram wins under paraphrase*) is invariant to dense scale: the Engram numbers barely move across our three sizes, and only the retrieval baselines drift as their answering-LM improves (Appendix P).

Our retrieval baselines are the retrieval step itself (top-3 sentence-encoder lookup over atomic facts), not the full Mem0 or MemMachine systems with their extraction and consolidation machinery; we skip those because our facts are already structured [Maharana et al., 2024, Wu et al., 2024].

Paraphrase generalization. The mechanism behind the paraphrase result is worth stating, because it is also a limit: the hash addresses a token-level suffix N-gram, so different surface forms of the same fact map to *different* rows. A fact written once therefore transfers to a paraphrase only when the paraphrase ends in the same tokens; writing the fact at every anticipated phrasing (multi-trigger insertion) is what closes the gap and drives the 96.9% vs. 75.0% win above. The contract is explicit—the trigger N-gram—rather than left to an encoder's notion of similarity. Single- vs. multi-trigger generalization is detailed in Appendix F, with per-paraphrase rank data in Appendix L.

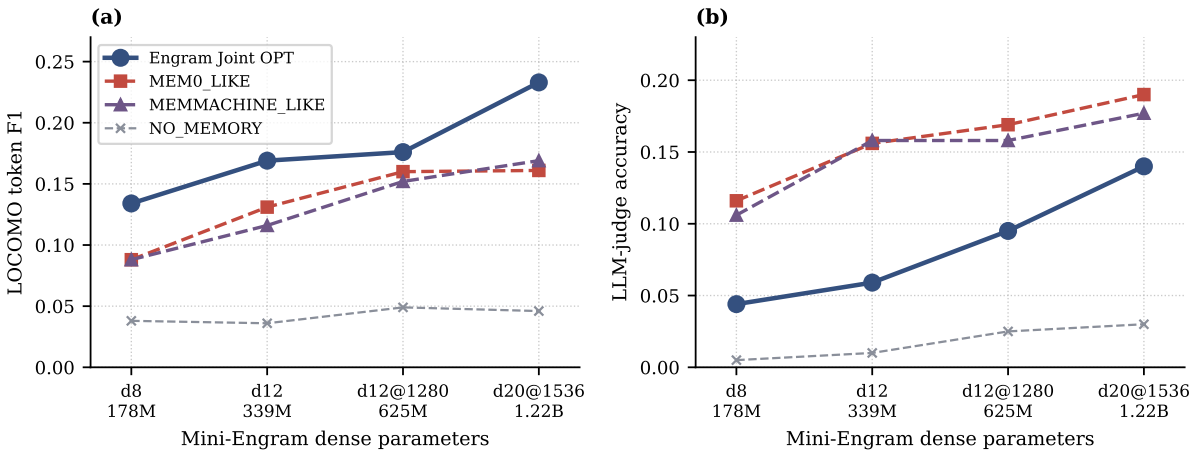


Figure 14. LOCOMO single-hop, full 10 conversations, scaling with Mini-Engram dense size. **(a)** *Token-F1*: User-as-Engram Joint OPT (blue) beats every retrieval baseline at every scale. **(b)** *LLM-judge accuracy* (Qwen2.5-14B-Instruct): the ranking flips—retrieval baselines (MEM0_LIKE, MEMMACHINE) beat Joint OPT by 0.05–0.10 because token-F1 over-credits Engram for the correct first answer token despite a noisy continuation.

5.5 LOCOMO single-hop fact recall

To evaluate the same memory systems on a published long-term conversational benchmark, we run all eight systems from Section 5.4 on LOCOMO [Maharana et al., 2024]. We use *all 10 LOCOMO conversations* and take the first 80 single-hop (non-adversarial) questions per conversation, for a total of ∼800 question-answer pairs grounded in dialog per model.

Setup. Retrieval baselines store the gold evidence turns; User-as-Engram stores the gold (question, answer) pair. Every system uses the same Mini-Engram-d12 to write the final answer, so the only difference is how the answer is found. We score token-F1 against the gold answer (details in Appendix A).

User-as-Engram Joint OPT clears every retrieval baseline on single-hop token-F1 (Figure 14). Absolute scores are low—Mini-Engram-d12 is a small base LM with no instruction tuning and token-F1 punishes verbosity—so the relative ordering is what matters. This is a best-case-for-each benchmark: retrieval stores the gold evidence sentence, Engram stores the gold (q, a) pair, and the only question is which extracts the answer more reliably under a fixed LM. Engram wins because the gate fires on the question's trigger N-gram and biases the next token toward the gold answer, whereas retrieval can miss the relevant turn and, even when it hits, the in-context evidence still competes with the base LM's priors.

Scaling at the optimal config. We repeated the same LOCOMO setup with all four trained Mini-Engrams at our ablation-optimal recipe (Section P.2); the Engram lead widens monotonically with dense size (Figure 14).

But the win depends on the kind of question. Across LOCOMO's four answerable categories, writing the fact wins on single-hop, multi-hop, and reasoning questions—where the stored question→answer pair is exactly what is needed—but loses on open-domain questions, where the answer is a long verbatim span of a sentence the model never saw and a one-token nudge cannot rebuild it (Figure 15 and Table 2). So User-as-Engram is a specialized tool, not a

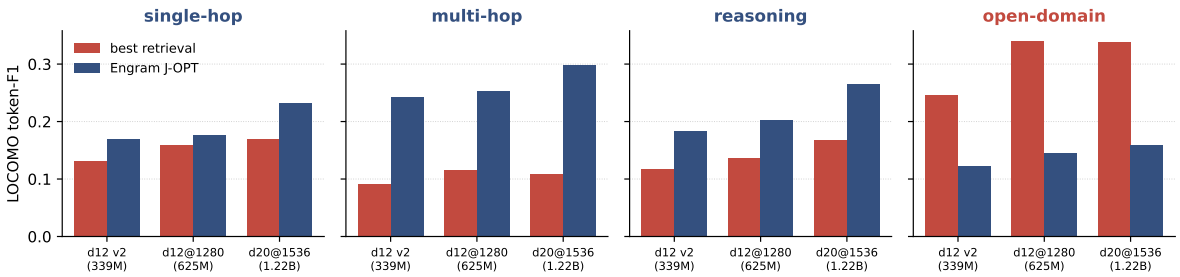


Figure 15. LOCOMO category breakdown (token-F1), Engram Joint OPT (blue) vs. the best retrieval baseline (red; max of MEM0_LIKE and MEMMACHINE_LIKE) across three dense scales. Engram wins single-hop, multi-hop, and reasoning at every scale—the per-fact (question, answer) loss encodes the question→answer map that retrieval must chain across evidence sentences—but loses open-domain, where the answer is a verbatim span of an unseen evidence sentence that a single-token bias cannot reconstruct.

Table 2. LOCOMO category × model token-F1. Engram Joint OPT vs. the best retrieval baseline (MEM0_LIKE or MEMMACHINE_LIKE) per cell. The Engram win is category-dependent (Figure 15).

category	answering LM	MEM0	MEMMACH	**Engram J-OPT**	lead vs. best retr.
single-hop	d12 v2	0.131	0.116	**0.169**	+29%
	d12@1280	0.160	0.152	**0.176**	+10%
	d20	0.161	0.169	**0.233**	+38%
multi-hop	d12 v2	0.092	0.076	**0.243**	**+165%**
	d12@1280	0.115	0.104	**0.253**	**+120%**
	d20	0.102	0.108	**0.299**	**+178%**
reasoning	d12 v2	0.117	0.113	**0.183**	+56%
	d12@1280	0.123	0.136	**0.203**	+49%
	d20	0.135	0.168	**0.266**	+58%
open-domain	d12 v2	**0.247**	0.209	0.122	−**51%**
	d12@1280	**0.341**	0.327	0.146	−**57%**
	d20	0.314	**0.338**	0.159	−**53%**

drop-in replacement for retrieval: a real system would route by question type, or use Engram for the facts it was taught and retrieval for free-form answers from the session.

One caveat about the score, and its fix. Token-F1 gives credit for partial word overlap, and our basic insertion only trains the *first* answer token, so it can score well by getting that first word right over an otherwise noisy answer. When we re-score with an LLM judge [Zheng et al., 2023] (Qwen2.5-14B, Appendix A), first-token Engram indeed slips behind retrieval on single-hop—but training the *whole* answer with multi-token Joint OPT fixes it and again overtakes the best retrieval baseline on the larger models. The paraphrase and storage wins above score only the first token, so they are unaffected. (The judge is stricter than LOCOMO's official one, so these numbers are not directly comparable to its leaderboard.)

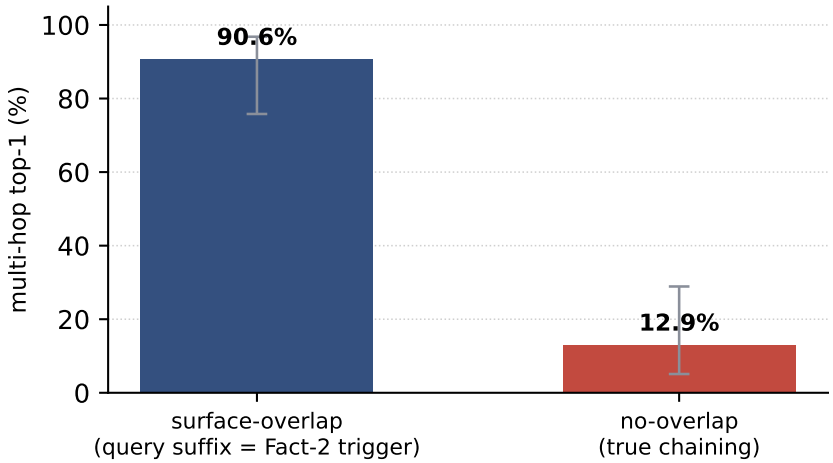


Figure 16. Multi-hop reasoning over Engram-inserted facts (n=63 chained pairs on Mini-Engram-d20, two OPT-15 insertions per item). Per-fact direct recall is 99.2% (both facts are insertable), so the gap is chaining, not insertion. When the query's suffix N-gram coincides with Fact-2's trigger, the gate fires and recall is 90.6%; when true chaining is required, it collapses to 12.9% (Wilson 95% CIs shown). The gate is a surface-N-gram hash—it matches, it does not compose.

5.6 Multi-hop reasoning over inserted facts

We probe chaining directly: store two facts ("my doctor is Patel", "Patel works at Globex") and ask a question that needs both ("where does my doctor work?"). On 63 such pairs, half worded so the question ends in the same words as the second stored fact and half not, we can separate a true chain from a lucky word-match.

The result is almost all-or-nothing (Figure 16): when the question happens to end in the second fact's words, recall is high; when a real chain is needed, it falls to near chance—and the few hits are word-match coincidences. Since each fact alone is recalled essentially perfectly, the gap is chaining, not storage. This is the limit the store shares with a per-user LoRA: it matches words, it does not reason across them. The recite-then-reason recipe (Section 8) is the natural fix.

6 Layered architecture: shared reasoning skill + per-user content

So far: per-user LoRA contaminates (Section 3) and per-user Engram preserves direct recall but lacks in-context reasoning skill (the per-user Engram alone reaches only 23% indirect_any at n=20 in Figure 17). Can the two be *composed* without inheriting LoRA's damage? Our thesis: personal memory splits into *content* (per-user, trigger-keyable, low-cost) and *reasoning skill* (shared patterns, learnable from held-out users); storing each the right way beats every all-in-one baseline on every measure. Two hypotheses make this concrete:

- **H1 (naive combination)**: the naive stack of per-user LoRA + per-user Engram inherits LoRA's indirect-reasoning damage; Engram's local edit does not rescue the composition.
- **H2 (layered architecture)**: one *shared* LoRA (cross-user reasoning skill) + per-user Engram (local content), the layered design, splits memory correctly. The shared LoRA's contamination is paid once and shared across all users, and the per-user Engram adds none on top (the layered design's Δbpb equals the shared LoRA's alone, by Section 4's locality property).

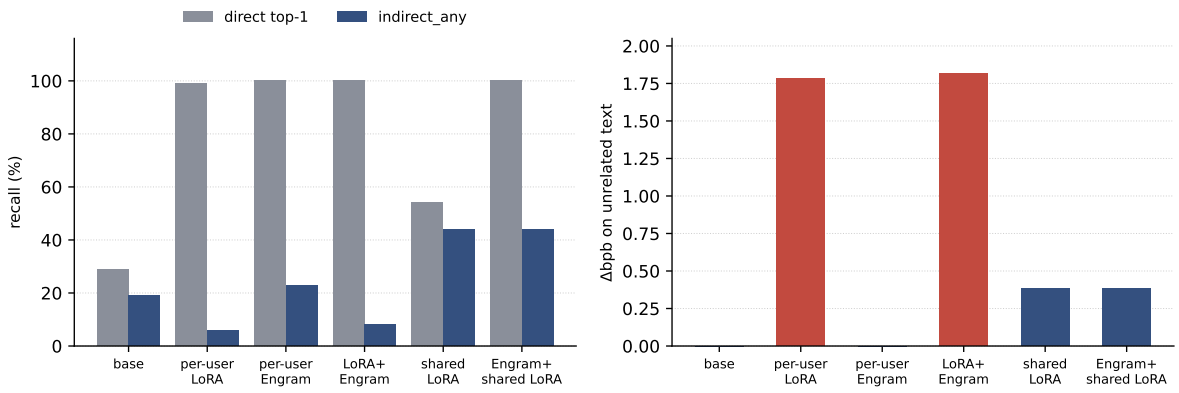


Figure 17. The six parametric conditions on n=20 test users (Mini-Engram-d20, seed S0; bars left-to-right as listed in the text). **Left:** the layered design (Engram + shared LoRA) matches per-user LoRA's direct recall (100% vs. 99%) while delivering 7.4× its indirect_any on this seed (44% vs. 6%); neither the per-user Engram (no skill) nor the shared LoRA alone (never saw the user's facts) suffices. **Right:** the layered design adds *zero* contamination on top of the shared skill (Δbpb +0.386, equal to the shared LoRA alone), while per-user LoRA and the naive LoRA+Engram stack pay +1.78/+1.82. The retrieval conditions appear in Figure 29; 3 seeds in Appendix O (3-seed mean: layered 41%, per-user LoRA 7%).

6.1 Design

Train a single shared LoRA on cross-user in-context-reasoning data: for each held-out training user (here, u020–u029), render each indirect QA as a completion-format sample "`Facts: <fact1>. <fact2>. ... Q: <q> A: <gold>`", using only the facts the question needs. This teaches the LoRA the *pattern* "given facts in context, do the reasoning," without memorizing any specific user's content. At inference, attach the shared LoRA + the test user's Engram-row override and ask the indirect question without facts in the prompt: the Engram provides content (via trigger N-gram lookup), the shared LoRA provides the skill (Figure 1).

6.2 Six-condition head-to-head

We compare six parametric conditions, shown as the six bars (in this order) in Figure 17: **the untouched base**, **per-user LoRA**, **the per-user Engram alone** (content, no shared skill), **the naive LoRA+Engram stack**, **the shared LoRA alone** (skill, no per-user content), and **the layered design** (per-user Engram + shared LoRA); the retrieval variants are introduced in Section 6.4. We evaluate every per-user combination on n=20 test users (n=10 training users held out for the shared LoRA), measuring direct top-1/top-5 recall on the user's facts (completion-format prompts), indirect top-1 and indirect_any on completion-format indirect probes ("`Q: <q>\nA:`", greedy 16-token continuation), and val_bpb on a held-out 262 K-token ClimbMix shard (Figure 17).

6.3 Results: the layered design wins on every measure

The headline is the layered-design-vs-per-user-LoRA contrast (Figure 17). The layered design matches per-user LoRA on direct recall (100% vs. 99%) but answers indirect questions 7.4× **more often** on this canonical seed (44% vs. 6%; 5.6× averaged over three seeds), with 4.6× **less damage to unrelated text**—all of it from the shared skill, none from the per-user Engram—and **not a single user out of 20 made worse than the untouched model** (against 17 of 20 for the

LoRA), all at 88 KB per user instead of 14.2 MB. The gap is stable across three seeds (paired-bootstrap 95% CI on the layered−LoRA indirect difference $[+31, +37]$ pp; per-seed spread in Appendix O).

H1 holds: bolting an Engram onto a per-user LoRA does not rescue its reasoning—the LoRA's damage dominates the combination. H2 holds on all three counts: adding the per-user Engram on top of the shared skill costs *zero* extra damage to unrelated text (its Δbpb equals the shared LoRA's alone), keeps direct recall at 100%, and roughly doubles indirect accuracy over either the Engram alone or the untouched model. Neither piece works on its own—the Engram has the facts but no reasoning, the shared skill has reasoning but has never seen the user's facts—and only together do they win.

6.4 Does RAG just close the gap instead?

The obvious objection to the layered design is: why not just retrieve the facts and put them in the prompt? We test this two ways (the naive-RAG and RAG+shared-LoRA conditions in the context-cost plot, Appendix I, Figure 29). On the Engram base itself, putting facts in the prompt sits *below* the layered design—the base LM never learned to use an in-context fact list, and even handing it the exact right facts does not help, because the missing piece is the reasoning skill, not the facts. The fairer test feeds the retrieved facts to a real instruction-tuned model (Qwen2.5-3B-Instruct, about 2.5× larger). Even there, the layered design—with no retrieval and an empty prompt—lands within a few points of the larger model, and adding retrieval on top of the shared skill passes it once context is free. The plain rule: no room in the prompt → use the layered design; room to spare → add retrieval on top of the shared skill.

6.5 RAG-vs-Engram trade-offs sharpen as the KB grows

The experiments above used each user's native 34 facts. Production fact counts are typically larger (10^2–10^3: chat history, calendar, preferences, contacts). We augment each test user's 34 facts with distractors sampled from the 30-user schema-family pool (`u000-u029` minus the active user; 1008 facts) to reach $N \in \{34, 100, 200, 300, 500, 1000\}$. Indirect probes are unchanged; the retriever now discriminates the gold fact among N candidates. The layered design is unaffected: the per-user Engram table holds only the test user's facts, independent of population size.

Retrieval degrades quickly: as the candidate pool grows, top-3 recall on `all-MiniLM-L6-v2` collapses several-fold purely from more candidates in the same embedding space, which drives the accuracy drop. Naive RAG, and even Qwen-3B + RAG, fall below the layered design once the KB passes ∼100 facts, while the layered design holds flat—its per-user table never grows. The clean reading (Figure 18): *which method wins shifts with deployment scale*. At a few dozen facts, RAG on a larger model is the accuracy ceiling and the layered design is the zero-context option; by a few hundred facts, the layered design decisively wins.

Where each side's failure comes from. The two limits are mechanistically distinct. Engram's density ceiling (Section 5.2) comes from forward-pass interference among co-active rows inside one user's table, and is bounded by per-user fact count. RAG's retrieval ceiling comes from nearest-neighbor confusion across an ever-growing candidate pool, and is bounded by whatever pool the retriever sees (per-user, per-tenant, or per-corpus). For 10^2–10^3 facts/user, Engram density is the limiting factor on the parametric side; for 10^4+ facts in a shared corpus,

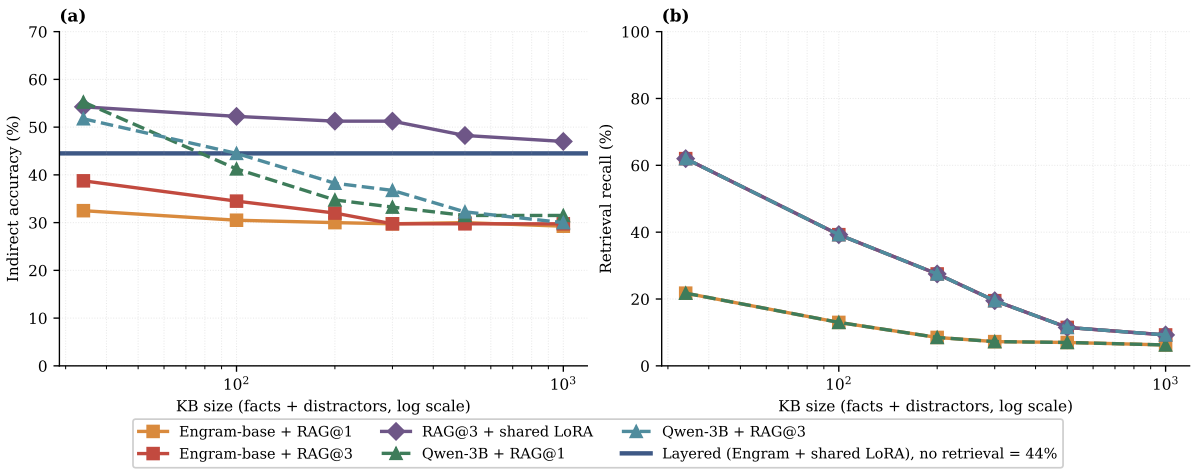


Figure 18. Indirect-reasoning accuracy and retrieval recall vs. KB size (n=20 users, 20 indirect probes each; KB = test user's 34 facts + distractors sampled from the 30-user schema-family pool, log x-axis, $N \in \{34, 100, 200, 300, 500, 1000\}$). **(a)** the layered design (horizontal solid line at 44%) is invariant to KB size; the retrieval-based conditions degrade monotonically. **Naive RAG and Qwen-3B + RAG fall below the layered design at KB $\geq$ 100; at KB=1000 the gap is 14 pp.** RAG top-3 + shared LoRA is the most robust to KB growth (54% $\rightarrow$ 47%) because the shared reasoning skill compensates when retrieval misses, though its lead over the layered design shrinks from 10 pp to 3 pp across the sweep. **(b)** Retrieval recall (fraction of probes where the retrieved set covers every fact the question needs) drops by 7$\times$ for top-3 (62% $\rightarrow$ 9%) between KB=34 and KB=1000, which is the mechanistic driver of (a).

retrieval is the limiting factor on the RAG side. The two costs scale on different axes and cross around $N \approx 100$.

7 Multi-tenant serving system

We implement `EngramServer`, a ∼50-line wrapper around an Engram-pretrained model. The base and global tables stay on the GPU; each user's small override map lives in CPU memory. A request resolves the user, writes their rows, runs the forward pass, and restores the originals—no router, no custom kernel, no graph rewrite. Writing and restoring the rows are cheap relative to the forward pass in every configuration we measured—a sub-millisecond array write on the ablation-optimal `d12e1280` (Figure 19), and at most ≈2.2 ms each in the earlier per-component breakdown on the smaller `d12` v2 checkpoint (Appendix H)—and the frozen forward pass dominates the rest.

It scales flat in the number of users. Figure 19 reports throughput, latency, storage, and recall across four deployment scales.

Per-request work does not depend on how many users share the server, so throughput and latency barely move as users triple (Figure 19); each extra user costs one small row-swap and no shared state. Two users are never in the table at the same time, so one cannot read another's facts: cross-user leakage is zero by construction. (The only residual overlap is coincidence—two users with the same favorite spice—and it is small.) By contrast, serving per-user LoRAs needs custom CUDA kernels and batch-routing (S-LoRA [Sheng et al., 2024], Punica [Chen et al., 2024]); here the write is just an array assignment. Batched multi-user serving would add

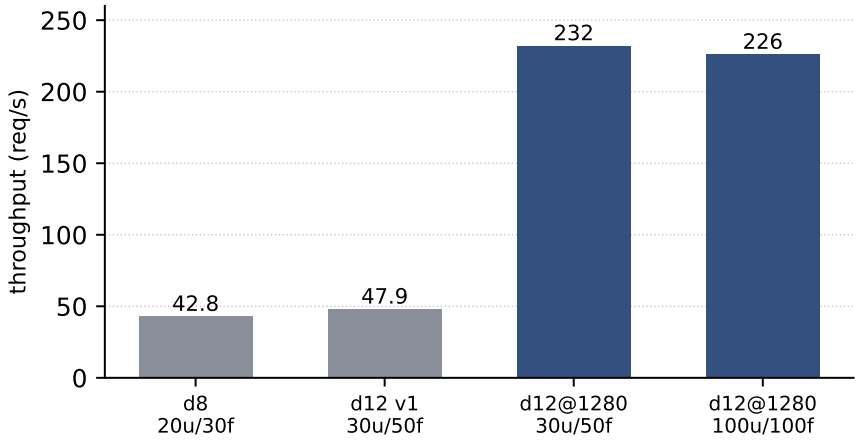


Figure 19. EngramServer throughput across four deployment scales on one idle Blackwell GPU. On the ablation-optimal d12@1280 the server reaches 232 req/s (30 u/50 f) and holds 226 req/s at 100 u/100 f—within 3% as tenants triple—at 4.4 ms p50 latency, and a sub-millisecond override apply (0.03 ms p50; the larger per-component figures in Appendix H are from an earlier, slower benchmark on the smaller `d12` v2 checkpoint), 88 KB/user, and 62%/96% top-1/top-5 own-fact recall.

one gather per Engram layer, which we do not benchmark.

8 Discussion and limitations

8.1 When to use what

Which method to use turns on three things: *how many facts each user has*, *whether the queries need indirect reasoning*, and *whether context budget or per-user storage is the tighter constraint*. Table 3 gives the recommended method for each case.

Table 3. Cheat sheet for which method to use. "invariant" = the number does not degrade with KB / population size.

case	recommended method	per-user cost	headline metric
<10 facts/user, direct recall only	ICL / strong-retriever RAG	0 KB persistent	near-perfect recall
10–1000 facts/user, indirect matters	layered design (Engram + shared LoRA)	**88 KB**, 0 ctx toks	44% indirect (invariant in KB)
10–1000 facts/user, ctx budget free	RAG top-3 + shared LoRA	88 KB + 44 ctx toks	54% indirect (KB=34)
enterprise, 100% direct top-1, <10K users	per-user LoRA (S-LoRA serving)	14.2 MB	100% direct, indirect cost
long-form / open-domain / shared corpus	RAG	0 KB persistent	wins LOCOMO open-domain +53%

The most important row is the second: the layered design is the right choice for the common case (10–1000 facts/user, indirect reasoning matters, many users), and decisively beats RAG with a $\sim$2.5$\times$ larger instruction-tuned LM at $N \geq 100$. *Any* method that reaches 100% direct top-1 pays a price in indirect reasoning; the question is whether that price is baked in (a global LoRA), manageable (a local Engram, where the trigger N-gram is the contract), or set by retrieval (RAG, where the encoder must pick the gold fact out of N candidates).

8.2 Open limitations

Shared multi-hop reasoning gap. Both LoRA and Engram are surface-trigger keyed; neither composes facts across triggers ("if my doctor is Patel and Patel works at Globex, what is my

doctor's employer?"). User-as-Engram inherits this gap from per-user LoRA. A candidate fix is to mix recite-then-reason traces into Engram pretraining; we leave it to future work. The follow-up suggests a second route: in Qwen3-8B, a single edit written in the first half of the network (layers 7–19) does support one further hop through pretrained knowledge, while the same edit in the last quarter does not (Section 4.4). A subject-keyed read placed earlier is therefore worth testing, though Section 4.3 shows that an Engram row written early is mostly washed out at our budget.

Locality is keyed on the trigger, not the subject. An Engram row is addressed by the trigger's last two or three tokens. Across users, the per-user override tables give zero leakage by construction. Within one table, two subjects whose facts end in the same N-gram ("… was born in") share rows, and we have not measured how much one such fact leaks to the other. The follow-up on Qwen3-8B compares a gate, a LoRA and a subject key, but does not include an Engram model; its multi-fact (10 and 50 facts per edit) and direct "does the unit fire on other people" measurements are still running, and its counter-example arm covers only layers 7, 15 and 23 (Appendix C.3).

Within-user density ceiling. Joint OPT at 1000 facts reaches 35% top-1. The slot space is highly sparse ($< 1\%$ occupancy), so the constraint is forward-pass interference among co-active rows, not hash collisions. Per-user scoped tables (load only the relevant rows for the current query, via a small classifier) is a candidate future fix; another is the multi-fact-in-the-loss recipe change in Section 8.3.

Engram pretraining required. Our method assumes an Engram-pretrained base. We trained four ourselves at 178 M, 339 M, 625 M, and 1.22 B total parameters; production deployment depends on either DeepSeek releasing weights or pretraining your own ($\sim$10 h on a single Blackwell GPU at our 625 M scale; production-scale Engrams require correspondingly larger compute budgets).

8.3 Future work: training-recipe changes

Three changes to the training recipe target the limits we documented, and we expect more from them per unit of compute than from simply making the model bigger. *(1) Put many facts in the loss during training*, so the model learns to hold several at once. We tested a lighter version of this by finetuning d12@1280, and it helps at high density—but only by reaching the same recall in fewer optimization steps, not by raising the ceiling (Appendix P.1, Figure 31). This pins the density ceiling on inference-time optimization, not pretraining. *(2) Train with random per-user offsets to the hash*, which would let the gate read rows it never saw in training and bring back a per-user-salt option. *(3) Mix in "recall the fact, then reason" examples*, the most direct attack on the chaining gap.

A negative result worth recording. We tried improving the shared skill by training it on reasoning traces from a teacher model. It cut the contamination but also *lowered* accuracy (Appendix P). The reason is instructive: those traces assume the model first writes out a thinking step, while our test reads only the final answer. The recipe and the way you score it have to be designed together—you cannot mix and match them. Remaining layered follow-ups:

a larger shared-skill training set, training the per-user table and the shared skill together, and starting from a chat-tuned base.

9 Related Work

Memory architectures and adjacent work. Engram [Cheng et al., 2026] sits in a long line of trainable key–value memory modules [Geva et al., 2021, Lample et al., 2019, He, 2024, Berges et al., 2025, Huang et al., 2024b, 2025, Yu et al., 2025, Pagnoni et al., 2025, Liu et al., 2025], but none of them addresses writing a single user's fact in at inference time. Stacking edits without a combiner echoes LoRA stacking for Stable Diffusion [CivitAI, 2024] (LoRAHub [Huang et al., 2023] instead trains one). A mirror-image line keeps memory *outside* the weights but executable—User as Code [Li, 2026] compiles a user's history into typed state and rules—which we contrast with our in-weights edits in Section 5.

Memory systems for LLM agents. A large body of work gives an agent long-term memory by keeping facts *outside* the weights and retrieving them at query time: paging systems [Packer et al., 2023]; extraction- and graph-based stores [Chhikara et al., 2025, Xu et al., 2025, Rasmussen et al., 2025, Li et al., 2025, Hu et al., 2026, Wang et al., 2026]; and a line that learns *how* to read and write the store [Yan et al., 2025, Yu et al., 2026, Wang et al., 2025], with surveys cataloguing the space [Zhang et al., 2024, Wu et al., 2025, Du, 2026]. None writes a fact *into* the model. (One concurrent system even shares our name [Patel & Patel, 2025] while doing the opposite—it orchestrates memories in natural language.) We use Mem0- and MemMachine-style retrieval as our strongest baselines (Section 5.4) and ask a different question: *when* does writing a fact into the weights beat keeping it in a retrievable store [Pollertlam & Kornsuwannawit, 2026]?

Personalizing language models. Adapting a model to an individual user is an active area in its own right [Zhang et al., 2024, Liu et al., 2025, Xu et al., 2026], with benchmarks such as LaMP [Salemi et al., 2024] measuring personalized generation. Most methods personalize through the prompt or a retrieval store; the per-user LoRA line above is the parametric alternative, and User as Engram is our attempt to win the locality of a prompt at the zero-context cost of a weight edit.

Retrieval and non-parametric memory. Retrieval-augmented generation [Lewis et al., 2020, Gao et al., 2023] is the dominant non-parametric route, spanning retrieve-then-read pretraining, large-scale retrieval pretraining, and self-reflective retrieval [Guu et al., 2020, Khandelwal et al., 2020, Borgeaud et al., 2022, Izacard et al., 2023, Asai et al., 2024]. Its quality is bounded by the encoder's ability to surface the right evidence among many candidates—exactly the failure mode we measure as the knowledge base grows (Section 6.5).

Personal memory benchmarks. Long-term personal memory is evaluated by LOCOMO [Maharana et al., 2024], LongMemEval [Wu et al., 2024], BEAM [Tavakoli et al., 2025] (multi-turn evolving beliefs), and PersonaMem-v2 [Jiang et al., 2025]. We use synthetic per-user fact corpora for the within-user fact-density sweeps (tighter control over the parameter we vary), and LOCOMO (Section 5.5) for end-to-end evaluation against retrieval baselines. We chose LOCOMO over LongMemEval and BEAM because its per-question evidence pointers let us

train a per-fact Engram row and a retrieval entry against the same gold sentence—an apples-to-apples "same gold, different method" comparison. LongMemEval and BEAM remain open for future work.

Knowledge editing and recitation. Knowledge editing changes a fact the model already holds—locating and rewriting weights [Meng et al., 2022, 2023, Dai et al., 2022, Yao et al., 2023], or routing around them [Mitchell et al., 2022a,b, Wang et al., 2024]—with ripple-effect benchmarks measuring the fallout [Zhong et al., 2023, Cohen et al., 2024, Meng et al., 2022b]. We do the opposite: we *add* facts the model never saw and want everything else left alone. Our follow-up (Section 4.4) uses ROME as the reference for a subject-keyed edit. Its low leak in layers 3–11 fits the account in which the subject's representation is enriched early and the attribute is read out later [Geva et al., 2023]. Where an edit is placed also changes what it does beyond the edited fact, which echoes the finding that causal localization does not tell you where editing works best [Hase et al., 2023]. Recitation methods, which surface a stored fact before reasoning over it [Wu et al., 2025, Sun et al., 2023], we reuse in Section 3.

Complementary learning systems and continual learning. The extra loss a per-user LoRA imposes on unrelated text is a modern instance of *catastrophic interference*: training on something new overwrites what was there before [McCloskey & Cohen, 1989], which decades of continual-learning work fight by protecting important weights [Kirkpatrick et al., 2017] or replaying old data, with the problem persisting for large language models [Shi et al., 2024]. The brain avoids it with a different architecture, and the word *engram* points to how: an engram is the physical trace a memory leaves in neural tissue [Semon, 1921], and *engram cells*—sparse neuron populations whose reactivation triggers recall—sit largely in the hippocampus [Tonegawa et al., 2015], while general skills live in the slow, distributed neocortex. Complementary learning systems theory [McClelland et al., 1995, Kumaran et al., 2016] argues this split is what lets a new episode be written without overwriting old skills. User as Engram is the same division made architectural: the per-user Engram row is the fast, sparse, local trace (the hippocampal engram); the shared LoRA and frozen backbone are the slow, distributed skill (the neocortex); and our measured locality (Δbpb $+0.00005$ on unrelated text, Section 3) is the engineering form of the *pattern separation* that keeps the two from interfering. The analogy is also honest about where our mechanism falls short: the hippocampus performs *pattern completion*—recovering a whole memory from a partial or indirect cue—whereas our gated trigger-N-gram lookup fires only on a near-exact surface match, which is exactly the multi-hop and paraphrase gap of Sections 5.6 and 5.4. The biology thus predicts the precise capability we still lack. Where continual learning asks one set of weights to absorb new facts gracefully, we sidestep the interference by writing each fact to its own address and leaving the shared weights untouched.

10 Conclusion

Personal memory is two jobs, not one: remembering a user's specific facts, and the general skill of reasoning with them. The standard in-weights recipe asks one per-user adapter to do both, and that is the root of the trouble: an adapter that must *hold* the facts cannot help also reshaping how the model thinks, so remembering and reasoning end up pulling on the same weights and the user pays for it in damaged knowledge and weaker reasoning. User as

Engram separates the two—a user's facts become a small, local edit, and the reasoning skill lives in one model everyone shares.

The deeper lesson is that the trouble is not unique to adapters. Every method that reaches near-perfect recall pays the same price in weaker reasoning over the stored facts; what differs is only *where* the price lands. A per-user LoRA pays it in the shared weights, on every query and stacking with every user; retrieval pays it in a search index that degrades as the candidate pool grows; User as Engram pays it at the one address its fact occupies, so the cost grows with a single user's facts and nothing else.

Our follow-up sharpens what "local" requires. A gate is not enough: an added neuron that is silent on most text still hands a new fact to other people as readily as a LoRA does. What keeps an edit to its subject is a key, as in ROME or Engram's address, or explicit training against counter-examples, which works for a LoRA as well but has to be repeated for every edit and costs reasoning. An Engram table gets the key from the architecture.

A century ago Richard Semon coined *engram* for the trace an experience etches into living tissue—a mark local enough that one memory does not erase the next. The brain pairs that sparse, local trace with a slow cortex that learns how to use it, and the pairing is what lets a person remember a new fact without forgetting how to think. Maya's assistant needs the same discipline. When we stop asking one set of weights to be both the memory and the mind, and instead give each user a small private trace beside a skill everyone shares, personalization stops fighting the model it runs on. That, more than any single number, is what *User as Engram* is for.

Acknowledgements

This paper was produced using Pine Copilot's voice-directed *whisper coding* workflow, in which the authors specify, discuss, and review the work by voice while a coding agent—Claude Code with Claude Opus 4.8—carries out the planning, coding, experiments, and paper writing. We thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

References

B. Li. User as code: Executable memory for personalized agents. arXiv:2606.16707, 2026.

R. Semon. *The Mneme*. George Allen & Unwin, London, 1921. (English translation of *Die Mneme*, 1904; origin of the term "engram").

J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

S. Tonegawa, X. Liu, S. Ramirez, and R. Redondo. Memory engram cells have come of age. *Neuron*, 87(5):918–931, 2015.

D. Kumaran, D. Hassabis, and J. L. McClelland. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.

X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. arXiv:2601.07372, January 2026.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. NeurIPS 2017.

T. Brown et al. Language models are few-shot learners. NeurIPS 2020.

A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners (GPT-2). OpenAI technical report, 2019.

S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? EMNLP 2022.

P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 2020.

Y. Gao et al. Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997, 2023.

N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. ICLR 2017.

D. Dai et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. arXiv:2401.06066, 2024.

S. Mangrulkar et al. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.

K. Jordan et al. Muon: An optimiser for hidden layers in neural networks. GitHub / blog post, 2024.

C. Huang et al. LoRAHub: Efficient cross-task generalization via dynamic LoRA composition. arXiv:2307.13269, 2023.

D. Wu et al. LongMemEval: Benchmarking chat assistants on long-term memory. arXiv 2024.

A. Maharana et al. LOCOMO: Evaluating very long-term conversational memory of LLM agents. ACL 2024.

M. Tavakoli, A. Salemi, C. Ye, M. Abdalla, H. Zamani, and J. R. Mitchell. Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs (BEAM). arXiv:2510.27246, 2025.

B. Jiang, Y. Yuan, M. Shen, Z. Hao, Z. Xu, Z. Chen, Z. Liu, A. R. Vijjini, J. He, H. Yu, R. Poovendran, G. Wornell, L. Ungar, D. Roth, S. Chen, and C. J. Taylor. PersonaMem-v2: Towards personalized intelligence via learning implicit user personas and agentic memory. arXiv:2512.06688, 2025.

A. Karpathy. nanochat: an experimental training harness for LLMs. https://github.com/karpathy/nanochat, 2026.

E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. ICLR 2022.

N. Houlsby et al. Parameter-efficient transfer learning for NLP. ICML 2019.

W. Su et al. Parametric retrieval-augmented generation (PRAG). arXiv:2501.15915, 2025.

Z. Tan et al. DyPRAG: Dynamic parametric RAG. arXiv:2505.19386, 2025.

J. Chen, H. Zhang, L. Pang, Y. Tong, H. Zhou, Y. Zhan, W. Lin, and Z. Zheng. Privacy-preserving reasoning with knowledge-distilled parametric retrieval-augmented generation (DistilledPRAG). arXiv:2509.01088, 2025.

Z. Tan, Q. Liu, and M. Jiang. Democratizing large language models via personalized parameter-efficient fine-tuning (OPPU). EMNLP 2024 (arXiv:2402.04401).

Y. Zhuang et al. HYDRA: Per-user adapters for personalised LLMs. arXiv 2024.

M. Bini, O. Bohdal, U. Michieli, Z. Akata, M. Ozay, and T. Ceritli. MemLoRA: Distilling expert adapters for on-device memory systems. arXiv:2512.04763, 2025.

R. Charakorn, E. Cetin, Y. Tang, and R. T. Lange. Text-to-LoRA: Instant transformer adaption. arXiv:2506.06105, 2025.

Z. Tan et al. PER-PCS: Per-user post-hoc LoRA composition. arXiv 2024.

Y. Sheng, S. Cao, D. Li, et al. S-LoRA: Serving thousands of concurrent LoRA adapters. arXiv:2311.03285, 2024.

L. Chen, Z. Ye, Y. Wu, et al. Punica: Multi-tenant LoRA serving. MLSys 2024.

G. Lample, A. Sablayrolles, M. Ranzato, L. Denoyer, and H. Jégou. Large memory layers with product keys. NeurIPS 2019.

P. He. PEER: Mixture of one million experts. arXiv 2024.

V. Berges, B. Oğuz, D. Haziza, W. Yih, L. Zettlemoyer, and G. Ghosh. Memory layers at scale. ICML 2025.

Z. Huang, Q. Min, H. Huang, D. Zhu, Y. Zeng, R. Guo, and X. Zhou. Ultra-sparse memory network (Ultra-Mem). arXiv:2411.12364, 2024 (ICLR 2025).

J. Huang et al. OverEncoding: hashed N-gram embeddings via averaging. 2025.

L. Yu et al. SCONE: scalable contextual N-gram embeddings. 2025.

A. Pagnoni, R. Pasunuru, P. Rodriguez, et al. BLT: byte latent transformer with hashed N-gram embeddings. arXiv:2412.09871, 2025.

A. Liu et al. SuperBPE: word-level BPE for compositional patterns. 2025.

CivitAI Community. LoRA stacking patterns for Stable Diffusion. https://civitai.com/, 2024.

K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT (ROME). NeurIPS 2022.

K. Meng et al. MEMIT: Mass-editing memory in a transformer. ICLR 2023.

Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. MQuAKE: Assessing knowledge editing in language models via multi-hop questions. EMNLP 2023.

R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models (RippleEdits). TACL 2024.

K. Meng et al. CounterFact: a counterfactual editing benchmark. 2022.

D. Wu, J.-C. Gu, K.-W. Chang, and N. Peng. Self-routing RAG: Binding selective retrieval with knowledge verbalization. arXiv:2504.01018, 2025.

Z. Sun et al. Recitation-augmented language models. ICLR 2023.

A. Yang, B. Yang, B. Zhang, et al. Qwen2.5 technical report. arXiv:2412.15115, 2025.

A. Yang, A. Li, B. Yang, et al. Qwen3 technical report. arXiv:2505.09388, 2025.

A. Grattafiori, A. Dubey, A. Jauhri, et al. The Llama 3 herd of models. arXiv:2407.21783, 2024.

A. Q. Jiang, A. Sablayrolles, A. Mensch, et al. Mistral 7B. arXiv:2310.06825, 2023.

DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437, 2024.

N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. EMNLP-IJCNLP 2019.

W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. NeurIPS 2020.

L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks. arXiv:2306.05685.

C. Packer, S. Wooders, K. Lin, et al. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.

P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. arXiv:2504.19413, 2025.

W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-MEM: Agentic memory for LLM agents. arXiv:2502.12110, 2025.

P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. Zep: A temporal knowledge graph architecture for agent memory. arXiv:2501.13956, 2025.

Z. Li, S. Song, H. Wang, et al. MemOS: An operating system for memory-augmented generation in large language models. arXiv:2505.22101, 2025.

S. Wang, E. Yu, O. Love, T. Zhang, T. Wong, S. Scargall, and C. Fan. MemMachine: A ground-truth-preserving memory system for personalized AI agents. arXiv:2604.04853, 2026.

C. Hu, X. Gao, Z. Zhou, et al. EverMemOS: A self-organizing memory operating system for structured long-horizon reasoning. arXiv:2601.02163, 2026.

D. Patel and S. Patel. ENGRAM: Effective, lightweight memory orchestration for conversational agents. arXiv:2511.12960, 2025.

S. Yan, X. Yang, Z. Huang, et al. Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. arXiv:2508.19828, 2025.

Y. Yu, L. Yao, Y. Xie, et al. Agentic memory: Learning unified long-term and short-term memory management for LLM agents. arXiv:2601.01885, 2026.

Y. Wang, R. Takanobu, Z. Liang, et al. Mem-α: Learning memory construction via reinforcement learning. arXiv:2509.25911, 2025.

Z. Zhang, X. Bo, C. Ma, et al. A survey on the memory mechanism of large language model based agents. arXiv:2404.13501, 2024.

Y. Wu, S. Liang, C. Zhang, et al. From human memory to AI memory: A survey on memory mechanisms in the era of LLMs. arXiv:2504.15965, 2025.

P. Du. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. arXiv:2603.07670, 2026.

N. Pollertlam and W. Kornsuwannawit. Beyond the context window: A cost-performance analysis of fact-based memory vs. long-context LLMs for persistent agents. arXiv:2603.04814, 2026.

A. Salemi, S. Mysore, M. Bendersky, and H. Zamani. LaMP: When large language models meet personalization. ACL 2024. arXiv:2304.11406.

Z. Zhang, R. A. Rossi, B. Kveton, et al. Personalization of large language models: A survey. arXiv:2411.00027, 2024.

J. Liu, Z. Qiu, Z. Li, et al. A survey of personalized large language models: Progress and future directions. arXiv:2502.11528, 2025.

Y. Xu, Q. Chen, Z. Ma, et al. Toward personalized LLM-powered agents: Foundations, evaluation, and future directions. arXiv:2602.22680, 2026.

E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. ICLR 2022.

E. Mitchell, C. Lin, A. Bosselut, C. D. Manning, and C. Finn. Memory-based model editing at scale. ICML 2022.

D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, and F. Wei. Knowledge neurons in pretrained transformers. ACL 2022.

Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.3, knowledge capacity scaling laws. ICML 2025. arXiv:2404.05405.

B. Li. Incompressible knowledge probes: Estimating black-box LLM parameter counts via factual capacity. arXiv:2604.24827, 2026.

P. Wang, Z. Li, N. Zhang, et al. WISE: Rethinking the knowledge memory for lifelong model editing of large language models. NeurIPS 2024.

Y. Yao, P. Wang, B. Tian, et al. Editing large language models: Problems, methods, and opportunities. EMNLP 2023.

M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.

J. Kirkpatrick, R. Pascanu, N. Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13):3521–3526, 2017.

H. Shi, Z. Xu, H. Wang, et al. Continual learning of large language models: A comprehensive survey. arXiv:2404.16789, 2024.

K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang. REALM: Retrieval-augmented language model pre-training. ICML 2020.

U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis. Generalization through memorization: Nearest neighbor language models. ICLR 2020.

S. Borgeaud, A. Mensch, J. Hoffmann, et al. Improving language models by retrieving from trillions of tokens. ICML 2022.

G. Izacard, P. Lewis, M. Lomeli, et al. Atlas: Few-shot learning with retrieval augmented language models. *JMLR*, 24(251):1–43, 2023.

A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. ICLR 2024.

M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value memories. EMNLP 2021.

X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. ACL 2021.

Q. Zhang, M. Chen, A. Bukharin, et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. ICLR 2023.

T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. NeurIPS 2023.

M. Geva, A. Caciularu, K. Wang, and Y. Goldberg. Transformer feed-forward layers build predictions by promoting concepts in the vocabulary space. EMNLP 2022.

M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in auto-regressive language models. EMNLP 2023.

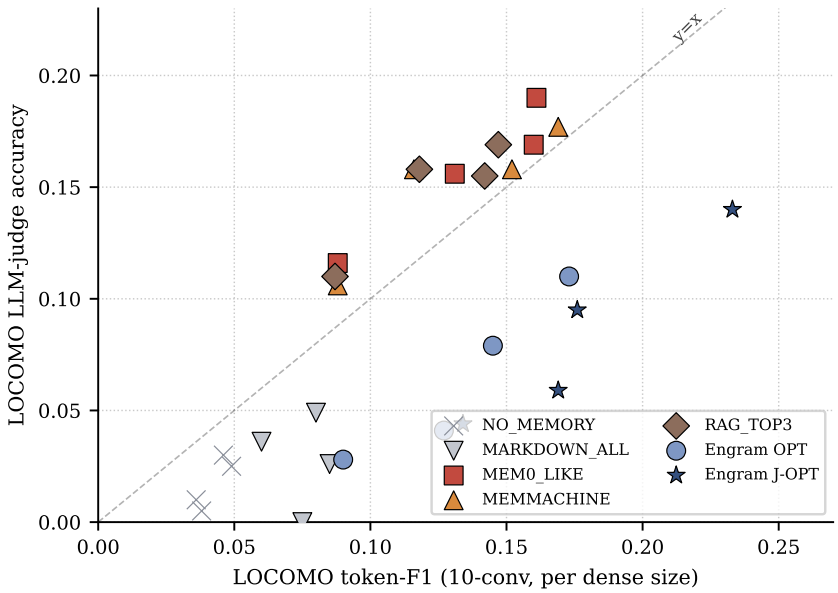


Figure 20. LOCOMO token-F1 vs. LLM-judge accuracy across all system × dense-size cells. Retrieval baselines (red/orange) sit near the $y{=}x$ line: their token-F1 and LLM-judge scores roughly agree. User-as-Engram OPT and Joint OPT (blue/cyan) sit systematically *above* the line, indicating that token-F1 over-credits Engram. The asymmetry is the metric mismatch documented in Section 5.5.

P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? Surprising differences in causality-based localization vs. knowledge editing in language models. NeurIPS 2023.

J. Kramár, T. Lieberum, R. Shah, and N. Nanda. AtP*: An efficient and scalable method for localizing LLM behaviour to components. arXiv:2403.00745, 2024.

S. Bills, N. Cammarata, D. Mossing, H. Tillman, L. Gao, G. Goh, I. Sutskever, J. Leike, J. Wu, and W. Saunders. Language models can explain neurons in language models. OpenAI, 2023. https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html.

L. Gao. Neural networks are not already clean. LessWrong, October 2026. https://www.lesswrong.com/posts/Dkh7pXKjoNWkkxQDz/neural-networks-are-not-already-clean.

Appendix A: LOCOMO diagnostics: token-F1 vs. LLM-judge

The category breakdown (Figure 15) and the underlying per-cell token-F1 for every system, scale, and category (Table 2) are in the body (Section 5.5). This appendix collects the two diagnostic figures behind the LLM-judge discussion of Section 5.5: Figure 20 shows that token-F1 over-credits Engram relative to an LLM judge, and Figure 21 shows that multi-token Joint-OPT closes the resulting judge gap.

Appendix B: Mechanistic analysis

This appendix consolidates the mechanistic evidence that is referenced piecewise in the main text (locality in Section 4.3, the multi-hop decomposition in Section 5.6) into a single account of

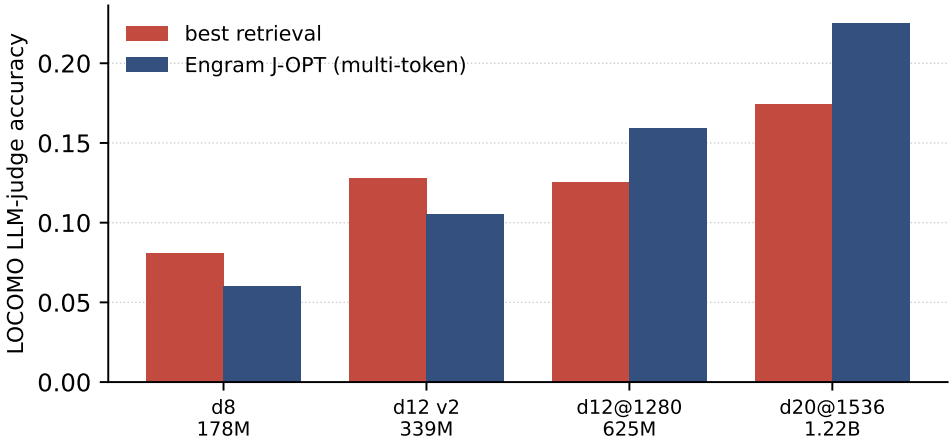


Figure 21. LOCOMO single-hop under an LLM judge (Qwen2.5-14B), matched multi-token pipeline. First-token OPT lets token-F1 over-credit Engram for a correct first token over a noisy continuation (Figure 20); training the *full* answer with multi-token Joint-OPT fixes this and overtakes the best retrieval baseline from d12@1280 upward (+27% at 625 M, +29% at 1.22 B). Below that cross-over the dense backbone is too small for the continuation to fall into place after the Engram-anchored prefix.

how a per-user fact becomes a retrievable, leak-free parametric edit. The picture is deliberately simple: User-as-Engram is not an opaque learned circuit but a **content-addressable memory** whose every step is either deterministic (hashing, gating) or directly measurable (the value the row writes, the gate it opens, the locality of its effect). We trace the path of one inserted fact through five observable stages, verifying each on the trained Mini-Engrams (d12@1280 and d20, 16 canonical facts). All numbers below are on the trained models unless stated.

(1) Addressing: a fact's trigger N-gram is a hash key. A fact is written at the row(s) that its trigger suffix N-gram hashes to, via the deterministic multiplicative-XOR hash of the Engram architecture [Cheng et al., 2026]. The address is a pure function of surface tokens—no learning, no per-user state—which is why two users' edits never interfere unless their *distinguishing* tokens collide. A collision audit on the randomly-initialized Engram demonstration code [Cheng et al., 2026] measures this directly: across 100 synthetic users $\times$ 30 facts, pairwise overlap on user-distinguishing key tokens (`Patel`, `Portland`, `1991`) is just 4.5%, and a 30-fact user occupies 0.027% of the demo table (the table is 99.97% empty after one user). Addressing is sparse and content-keyed, so capacity is never the limiting factor (Section P.2).

(2) The write opens its own gate and injects exactly its value path. A row reaches the residual stream only through the gated value $\alpha_t W_V e$, so a write has to do two things, and we can watch both. *It opens its own gate.* Because the gate's key is $W_K e$, writing the row raises the trigger-position gate from $\alpha \approx 0.015$ (an untrained synthetic trigger barely fires) to $\alpha \approx 0.99$ under the deployed OPT strategy (0.59 under closed-form UNEMBED_P), while every non-trigger position stays at $\alpha \approx 0.03$–0.04 (identical on d12@1280 and d20). *It injects its value path.* The residual change the deployed OPT row induces at the trigger has cosine **0.998** (d12@1280) / **0.999** (d20) with the analytic $W_V e$ projection—the gate and short-convolution scale the injection but do not redirect it. This reproduces, on the trained model and for the strategy we deploy, the read/write existence proof on the random-init demonstration code

(where writing a marker into one trigger's rows moved the trigger position by 116.96 vs. a mean 0.65 elsewhere, cosine 0.998 to the predicted W_V projection). (The small-magnitude closed-form marker is dominated by the convolution nonlinearity, so its *direction* is noisier; OPT drives the row norm up, 99 $\rightarrow$ 132, until the value path dominates.) The trained model corroborates this from the behavioral side via the sensitivity asymmetry (Section 5.1): ablating the Engram pathway degrades *factual* top-5 recall (93.3% retained) while *reading* comprehension is untouched (100% retained)—a 6.7 pp asymmetry that localises factual content to the Engram pathway.

(2b) What the row *encodes* vs. what it *optimizes*. A subtler reading concerns the row's relation to the gold token. The closed-form UNEMBED_P row points its value path straight at the gold token's unembedding (cosine **0.59–0.65** to lm_head$[y]$): "the row stores the gold value" is then literally true. Gradient OPT and Joint-OPT trade some of that direct alignment (cosine drops to 0.16–0.24) for higher recall, shaping the *full* next-token head rather than only the gold unembedding direction. The interpretability is therefore in the exact value-path injection (stage 2), not in the row equalling a single unembedding vector—the deployed row is a value the network reads cleanly, optimized against the true objective.

(3) Locality is exact—and it comes from addressing, not from the gate. It is tempting to attribute locality to the gate ("it fires only on the trigger"), but the gate measurement above shows the gate does *not* single out an arbitrary synthetic trigger before the write. Locality has a simpler source: we write only the trigger's rows, and the trigger's suffix N-gram is unique in the sequence, so every other position reads unchanged rows and its output is bit-identical. We verify the consequence not on one fact but across *all 16 canonical facts and both insertion strategies*: the maximum residual change over every non-trigger position and every layer is **0.000**, and the change before the Engram layer is **0.000**—exact, through every subsequent attention/MLP layer (the trigger position itself moves by order 1). Figure 7 contrasts this with a per-user LoRA fit to the *same* fact, whose effect is nonzero at every position and every layer and which perturbs unrelated text by mean 107. Because the edit is *addressed*, a per-user write contaminates no unrelated forward pass (Δbpb $+0.00005$ vs. a global LoRA's $+1.78$; Section 3), and cross-user leakage is zero by design (Section 4). Appendix C adds the converse check in a dense model: a learned gate alone does not localize a fact to its subject.

(4) Depth: the edit lands where the model has already "deepened". Engram rows are read at a late Engram layer, after the network has effectively committed to a next-token distribution. The LogitLens trace (Figure 22) shows Mini-Engram-d8 converging to its output distribution *faster* than the base—layer-3 KL is 3.66 lower—reproducing Cheng et al.'s "effective deepening" at our scale. A row therefore overrides a near-final prediction at exactly the position where it is decoded, rather than steering an early, still-malleable representation (the place where a global LoRA change does its damage). We make this dependence *causal* rather than merely correlational by re-running the insertion at the model's *early* Engram layer instead of the late one (layer 2 vs. layer 11 on d20, layer 2 vs. 7 on d12@1280; identical OPT-15 budget): top-1 recall collapses from **1.00 at the late layer to 0.25** (d20) / **0.31** (d12@1280) **at the early layer**, because an early injection must survive the rest of the transformer stack and largely washes out. The depth at which the Engram reads is not incidental—it is where a value-path edit can

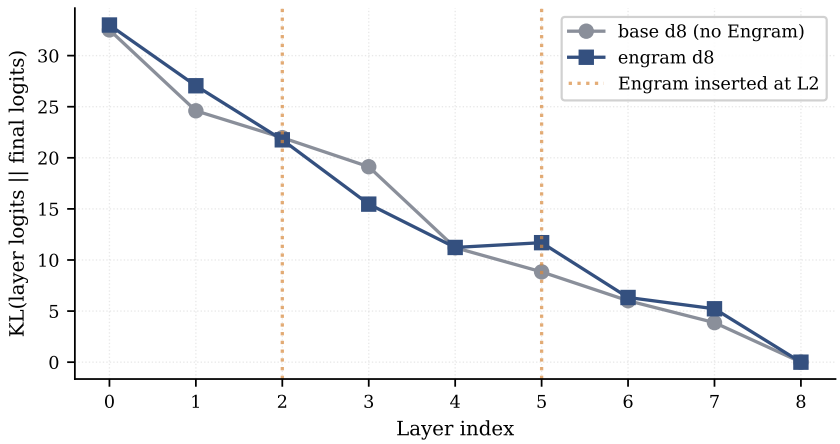


Figure 22. LogitLens KL by layer (Mini-Engram-d8 vs. base-d8). Engram converges faster at layer 3 (–3.66 KL gap), confirming Cheng et al.'s effective-deepening claim at our scale (stage 4 of the mechanism above).

override the prediction.

(5) Limit: the gate is a surface-N-gram hash, so it cannot compose across triggers. The same mechanism that makes the edit local also bounds what it can do. The multi-hop decomposition (Section 5.6, Figure 16) is the cleanest mechanistic readout in the paper: on a balanced 63-pair corpus, chained queries succeed **91%** of the time when the query's suffix N-gram *overlaps* the second fact's trigger, but only **13%** ($\lesssim$10% after filtering first-fact-wins coincidences) when true composition is required—even though per-fact direct recall is 99.2% in both subsets. The near-bimodal split is exactly what a surface-keyed lookup predicts: the gate fires on a matching trigger and returns the stored token, or it does not fire and the gold token is far from the top. There is no chaining circuit to find because, by design, there is none.

Synthesis. The five stages compose into one sentence: *User-as-Engram hashes a fact's surface trigger to a sparse row whose write both opens its own gate (α 0.02 $\rightarrow$ 0.99) and injects exactly its value path (cosine 0.999 to the predicted projection), at a late, already-deepened layer (early insertion recall collapses 1.00$\rightarrow$0.25), with the change exactly 0.000 at every other position—and therefore retrieves single facts with zero contamination but cannot compose across facts.* This is why the method cleanly separates *content* (the rows) from *reasoning skill* (composition, which must come from the shared LoRA in the layered design of Section 6)—the content/skill split the paper proposes is not an engineering convenience but a direct consequence of the mechanism.

Appendix C: Mechanism follow-up: Engram vs. LoRA, and a gate vs. a key

This appendix gives the methods and numbers behind Section 4.4 and the new paragraph in Section 3. Every record, script and a table mapping each number in the paper to its record and field are in `results/mechanism_followup/` and `code/mechanism_followup/` of the repository; `summarize.py` recomputes all of them on a CPU from the committed records. Means are over facts unless stated, and bracketed ranges are 95% bootstrap CIs over facts.

Table 4. Engram rows vs. per-user LoRA on the same 474 facts (Mini-Engram-d20, 20 users). Residual changes are means over facts (LoRA) or maxima over facts (Engram); patching values are medians.

	Engram (Joint OPT)	per-user LoRA
top-1 recall	474/474	474/474
residual change entering blocks 0–11, trigger	0 (all facts)	0.46 (block 1) → 1.22 (block 11)
residual change at block 19, trigger	0.67	1.36
non-trigger positions, any block	0 for 473/474; max 0.019	0.53 (block 1) → 1.30 (block 19)
three unrelated sentences, any block	0	0.37 (block 1) → 1.23 (block 19)
share of Δm attributed to MLP neurons	0.82	0.25
effect left, own top-10 / 100 / 1000 neurons reset	0.70 / 0.42 / 0.23	0.70 / 0.40 / 0.14
effect left, other method's top-10 / 100 / 1000 reset	0.79 / 0.60 / 0.45	0.83 / 0.61 / 0.49
effect left, 100 random neurons reset	1.00	1.00
top-100 neuron overlap (Jaccard, median)	0.24 (≈39 of 100 shared)	
$\cos(\Delta a_{\text{Engram}}, \Delta a_{\text{LoRA}})$, blocks 11–18 / 19	0.15–0.32 / 0.61 (shuffled facts: 0.01–0.11 / 0.40)	

C.1 Same facts as Engram rows and as a LoRA (Mini-Engram-d20)

Setup. Mini-Engram-d20@1536, the 20 headline test users (u000–u019). For each user, the facts are turned into (trigger, single-token gold) pairs exactly as in the head-to-head locality script, and we keep the 474 facts whose gold the base does not already predict. Each user is written twice from the same base: Engram Joint OPT (1,500 steps, lr 0.5) into the late Engram table (layer 11), and the repository's per-user LoRA (rank 64, $\alpha = 128$, $Q/K/V$, 1,500 steps, lr 5×10^{-4}). The metric is $m = \text{logit}[\text{gold}] - \text{logit}[\text{base top-1}]$ at the trigger. We record the relative change $\|\Delta x\|/\|x\|$ of the residual entering each block at the trigger, averaged over the prompt's other positions, and on three fixed unrelated sentences. MLP neurons are attributed by activation $\times$ gradient of m at the trigger [Kramár et al., 2024]. For patching, a set of neurons is reset to its base activations at the trigger, and we report the fraction of the method's Δm that remains (1 means no effect, 0 means the whole effect is removed). Restoring the saved Engram rows after the LoRA pass reproduces the Engram metric exactly (max $|\Delta m| = 0$).

The single fact with a non-zero off-trigger change (u010, emergency contact's phone) moves by at most 1.9% of the residual norm. The most likely cause is that one of its non-trigger positions reads a row written for another of the same user's facts: Joint OPT writes all of a user's triggers into one table, and those triggers share short N-grams. In the same run we also re-ran the six-condition head-to-head of Section 6 with the repository command (default users and evaluation tokens). It reproduces the headline: the layered design reaches direct recall 1.00 and indirect 0.44 ("any" metric), against 0.97 and 0.05 for per-user LoRA, with Δbpb $+0.39$ vs. $+1.59$ (record `T2/partB_layered_repro.json`).

C.2 Where a per-user LoRA stores a fact (Qwen3-8B)

Setup. Qwen3-8B, the same 20 users, the repository's per-user LoRA recipe (POLAR NTP mix, rank 64, $\alpha = 128$, lr 10^{-4}, 15 epochs) in two variants: $Q/K/V$ (the default) and MLP (gate/up/down). For every direct fact the LoRA gets right and the base gets wrong (up to 100 per user), the target is the first answer token where the base's teacher-forced prediction is wrong, and recovery is $(m_X - m_{\text{base}})/(m_{\text{LoRA}} - m_{\text{base}})$. We switch the LoRA on only in layers $\geq \ell$ (suffix), only in layers $< \ell$ (prefix), or only in one 4-layer band; ℓ_{50} is the last ℓ whose suffix still recovers half the effect. "Answer-writing" neurons are ranked statically by $|w_{\text{down}} \cdot (W_U[\text{gold}] - W_U[\text{pred}])|$ in the base [Geva et al., 2022]. Indirect questions use the

Table 5. Where a per-user LoRA stores a fact in Qwen3-8B (20 users).

	$Q/K/V$ LoRA	MLP LoRA
facts analysed	650	649
direct recall, base → LoRA	0.008 → 0.999	0.008 → 0.997
indirect accuracy, base → LoRA	0.11 → 0.30	0.11 → 0.35
median ℓ_{50} (of 36 layers)	20	20
best single 4-layer band, recovery	0.20	0.23
MLP share of the LoRA's direct logit effect	0.72	0.87
neurons holding 50% of the neuron contribution (median)	6	9
share held by top-100 answer-writing neurons (random 100)	0.28 (0.0002)	0.20 (0.0001)
effect left after resetting top-100 contributing neurons (random)	0.09 (1.00)	0.27 (1.00)
indirect acc., facts shallower vs. deeper than median ℓ_{50}	0.37 vs. 0.17	0.58 vs. 0.06
Spearman(ℓ_{50}, indirect correct); base-wrong only	-0.06; -0.05	-0.31; -0.28

repository's indirect set; a question's depth is the mean ℓ_{50} of the facts it needs.

No single band carries the effect (at most 0.23). The effect is built up across the middle layers and leaves through a handful of late neurons that already write answer tokens in the base. The depth–reuse association is clear for the MLP adapter but weak for the $Q/K/V$ adapter, and it is correlational: deeper storage and harder questions may share a cause.

C.3 A gate vs. a key: single-fact edits in Qwen3-8B

Setup. 200 fictional people, each with one fact "⟨name⟩ lives in the city of ⟨city⟩", using 55 real cities whose country the base already knows. Each edit stores one fact at one layer $L \in \{3, 7, \ldots, 35\}$, trained on the direct template with five prefixes (at most 100 Adam steps, early stop at target NLL < 0.05, lr chosen once on 5 dev facts). Methods: *neuron*, one new SwiGLU unit (gate row, up row, down column) appended to the MLP; *neuron_bias*, the same with a trainable gate bias (indistinguishable from *neuron* on every metric); *rank-1 LoRA* on down_proj or on up_proj (the latter passes through the existing SiLU gates); *ROME* [Meng et al., 2022], the closed-form rank-1 edit keyed on the subject's covariance-whitened MLP input; and a rank-16 $Q/K/V$ LoRA on every layer (the repository-style per-user LoRA). *+reg* adds KL(base‖edit) on 16 other people with the same relation, 8 other-relation prompts and 2×128 tokens of generic text, for a fixed 100 steps. Metrics: greedy accuracy on the direct prompt, on 3 held-out rewordings, and on 2 two-hop templates asking for the city's *country* (never trained); *leak*, the probability of the city's first token when asking about 40 other people (no edit: 4.6×10^{-4}); *fire10*, the fraction of 4,096 WikiText-103 test tokens on which the edit's unit exceeds 10% of its activation on the fact prompt; and *resid-move*, the fraction of those tokens whose residual at the edited layer moves by more than 1%. The fact given in context scores 0.66 on rewordings and 0.52 two-hop. Edits are trained in batches, each with its own parameters; Adam is element-wise, so this matches one-at-a-time training, which we checked.

Two further observations. Every subject-specific edit (ROME, and any method with +reg) shifts the *same* person's other facts more than a plain edit does (neighbor KL 1.3–1.7 vs. about 1.0). The change concentrates on the subject, which a per-user store would see as interference among that user's own facts. Second, leak is measured at the model's output on other people's prompts. Whether the neuron itself stays silent on those prompts was not recorded in this sweep; a supplementary run that measures it directly, and the 10- and

Table 6. Unregularized single-fact edits in Qwen3-8B, 200 facts per cell. Direct recall is ≥ 0.98 everywhere except ROME at L35 (0.00: ROME writes at the subject's last token, and after the final layer nothing carries it to the answer position).

	leak to other people				two-hop			fire10 on text	
layer	neuron	$\text{LoRA}_{\text{down}}$	LoRA_{up}	ROME	neuron	$\text{LoRA}_{\text{down}}$	ROME	neuron	$\text{LoRA}_{\text{down}}$
L3	.211	.007	.009	.005	.33	.33	.41	.561	.271
L7	.347	.094	.101	.003	.29	.32	.49	.016	.002
L11	.431	.447	.394	.029	.32	.31	.61	.005	.033
L15	.618	.647	.486	.135	.40	.39	.57	.011	.056
L19	.699	.759	.684	.185	.42	.45	.48	.053	.363
L23	.718	.793	.722	.178	.32	.36	.27	.045	.718
L27	.591	.685	.623	.299	.04	.03	.02	.047	.783
L31	.589	.563	.533	.155	.00	.00	.00	.036	.518
L35	.690	.859	.649	–	.00	.00	–	.273	.938
mean L3–31	.525	.499	.444	.124					

Table 7. Counter-example training (+reg), plain $\rightarrow$ +reg, 200 facts per cell (the $Q/K/V$ +reg arm had finished $\sim$170 of 200 facts when this was written). ROME at L7 for reference: leak .003, reworded .59, two-hop .49, fire10 3×10^{-5}, resid-move .09.

method	layer	leak	reworded	two-hop	fire10	resid-move
neuron	L7	.347 $\rightarrow$ .005	.44 $\rightarrow$ .45	.29 $\rightarrow$ .27	.016 $\rightarrow$ .027	.31 $\rightarrow$ .32
	L15	.618 $\rightarrow$ .015	.47 $\rightarrow$ .33	.40 $\rightarrow$ .20	.011 $\rightarrow$ .013	.19 $\rightarrow$ .20
	L23	.718 $\rightarrow$ .035	.39 $\rightarrow$ .34	.32 $\rightarrow$ .09	.045 $\rightarrow$ .068	.32 $\rightarrow$ .36
$\text{LoRA}_{\text{down}}$	L7	.094 $\rightarrow$ .003	.44 $\rightarrow$ .54	.32 $\rightarrow$ .33	.002 $\rightarrow$ .001	.20 $\rightarrow$ .26
	L15	.647 $\rightarrow$ .004	.43 $\rightarrow$ .25	.39 $\rightarrow$ .13	.056 $\rightarrow$ .068	.71 $\rightarrow$ .67
	L23	.793 $\rightarrow$ .076	.36 $\rightarrow$ .32	.36 $\rightarrow$ .04	.718 $\rightarrow$ .517	.99 $\rightarrow$ .94
LoRA_{up}	L7	.101 $\rightarrow$ .003	.44 $\rightarrow$ .52	.34 $\rightarrow$ .33	.296 $\rightarrow$ .165	.77 $\rightarrow$ .83
	L15	.486 $\rightarrow$ .004	.38 $\rightarrow$ .26	.26 $\rightarrow$ .14	.204 $\rightarrow$ .232	.86 $\rightarrow$ .86
	L23	.722 $\rightarrow$ .085	.33 $\rightarrow$ .29	.27 $\rightarrow$ .01	.340 $\rightarrow$ .370	.90 $\rightarrow$ .89
$Q/K/V$ r16	all	.495 $\rightarrow$ .016	.46 $\rightarrow$ .64	.18 $\rightarrow$ .12	–	–

50-facts-per-edit conditions, were still running when this was written and are not reported here. The two-hop setting differs from Section 5.6: here the second hop (city $\rightarrow$ country) is knowledge the model already has, which favors composition.

C.4 Does Engram change the base model's neurons?

If Engram took over factual lookups, the backbone's MLP neurons might become easier to describe. We audited 800 random live neurons (100 per layer) in each of two d8 models trained identically for 524M tokens, one dense (`base_d8`) and one with Engram at layers 2 and 5 (`engram_d8`). An LLM labeller (MiMo-v2.6-pro) wrote a label for each neuron from its top contexts, in the style of Bills et al. [2023]. We then applied a stricter test following Gao [2026]: the same LLM simulates the neuron's activations on held-out text from the label alone, we substitute the simulated activations into the model, and we measure the fraction of the

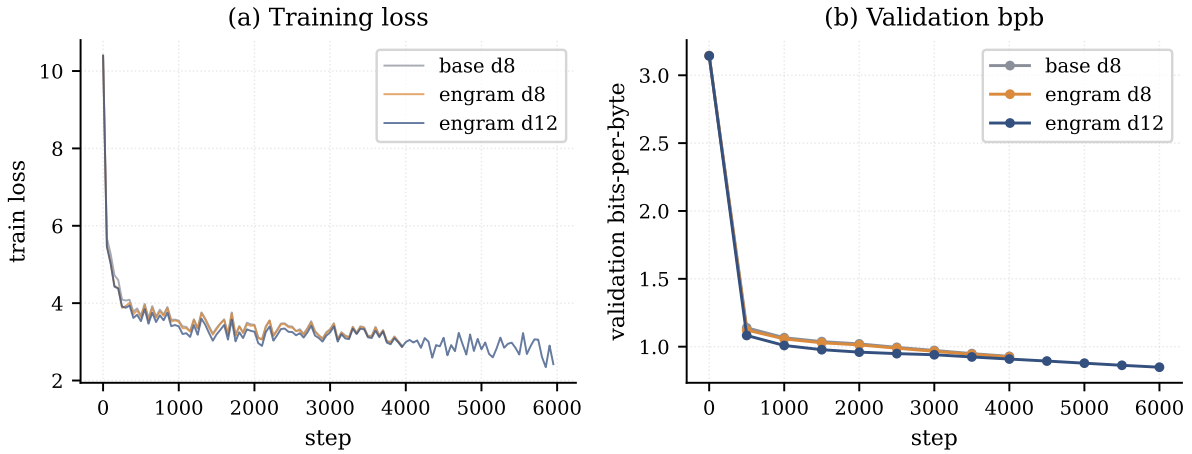


Figure 23. Mini-Engram pretraining curves. Engram d8 reaches lower validation bpb than base d8 at iso-FLOPs (matching the paper's finding at much smaller scale); engram d12 is substantially better thanks to the larger backbone and longer training.

next-token damage of zero-ablation that the substitution avoids. No measure differs: "one job" labels 0.96 in both (difference 0.00 [−0.02, 0.02]); median simulation correlation 0.43 vs. 0.46 (difference 0.03 [−0.03, 0.08]); median calibrated damage avoided 0.15 vs. 0.14 (difference −0.01 [−0.04, 0.02]); and entity/fact-type neurons are about 1% of neurons in both. The layers directly after the Engram injection lean slightly worse, also within noise. At this scale, then, Engram neither helps nor harms the interpretability of the rest of the network. The test can detect differences of roughly ±0.04–0.06. It used one labeller and no human check, and the d8 models hold little factual content for Engram to absorb.

Appendix D: Pretraining curves

Figure 23 shows the Mini-Engram pretraining runs that every later insertion experiment builds on. Adding the Engram table is not a tax on language modeling: at iso-FLOPs the d8 Engram model reaches marginally lower validation bits-per-byte than the d8 baseline (0.924 vs. 0.929 at step 4 000), reproducing Cheng et al.'s effective-deepening finding at our much smaller scale, and the larger d12 Engram model—more backbone, longer training—is substantially better (0.849 bpb). These checkpoints are the base models for the row-insertion results in the body.

Appendix E: Insertion strategy comparison

Figure 24 compares five ways of writing a fact into Engram rows at the hardest density we test (100 facts/user, Mini-Engram-d12). A random marker (RANDOM, a control) and the token input embedding (WTE) never recover the fact (0% top-1), confirming the recall signal is not an artefact of the metric. The closed-form pseudo-inverse marker (UNEMBED_P) reaches only 6%/18% top-1/top-5: it orients the row correctly but the gate suppresses its magnitude. Fifteen steps of per-row gradient descent (OPT-15) lift this to 38%/44%, and Joint OPT—optimizing all touched rows together against the user's full fact set—reaches 68%/96%, recovering most of what a per-user LoRA achieves at this density (Figure 11) at 161× less storage. The earlier single-fact-at-a-time comparison on the original 16-fact USER+ORG benchmark (Mini-Engram-

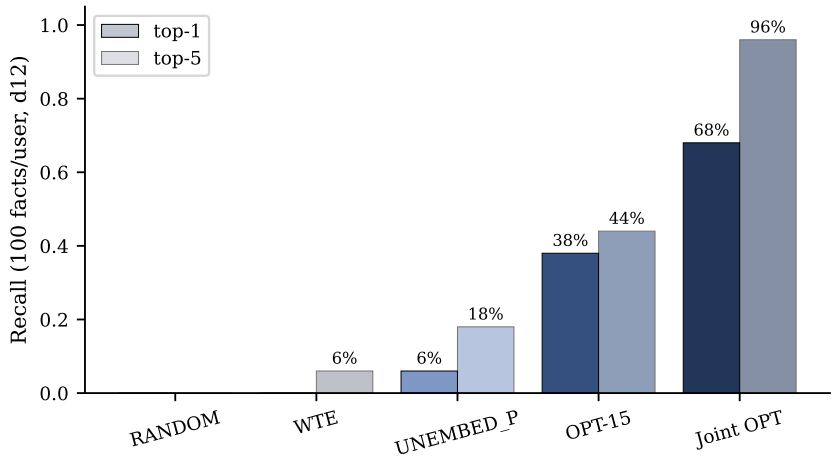


Figure 24. Top-1 and top-5 recall at 100 facts/user, Mini-Engram-d12. RANDOM (control) confirms our signal isn't noise; UNEMBED_P alone is too weak; OPT-15 helps; Joint OPT closes most of the gap to full LoRA fine-tuning at 161× less storage.

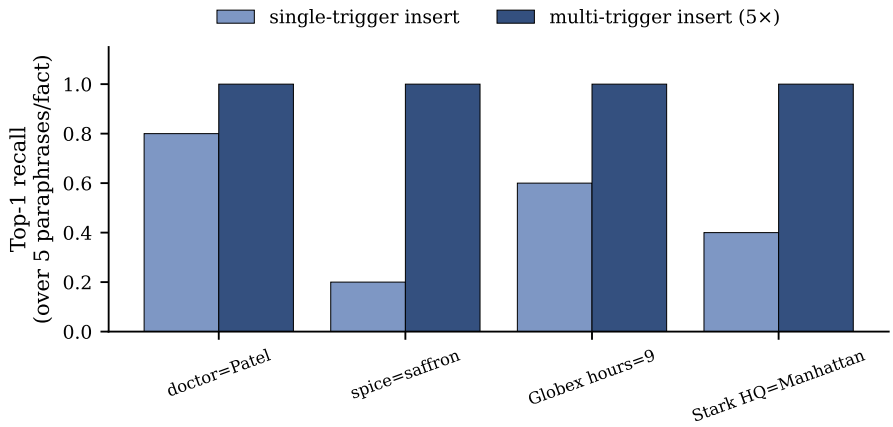


Figure 25. Paraphrase generalization across 4 facts × 5 paraphrases each. Single-trigger insertion gets 50% top-1 for free (suffix N-gram overlap); multi-trigger insertion (insert at all 5 paraphrases) gets 100% at 5× the per-fact OPT cost.

d8) is reported in Appendix K.

Appendix F: Paraphrase generalization

Figure 25 tests whether a fact written at one trigger phrasing answers the same question asked differently, across five held-out paraphrases per fact. Single-trigger insertion (write once, at the canonical phrasing) averages 50% top-1: 4/5 for the "doctor" fact, 3/5 for "Globex hours", 2/5 for "Stark HQ", and 1/5 for "spice". Generalization tracks suffix N-gram overlap—paraphrases that end in the same tokens as the trained trigger transfer for free, while divergent surface forms (e.g. "I love the spice" vs. "my favorite spice is") do not. Multi-trigger insertion (write the fact at all five phrasings, at 5× the per-fact OPT cost) drives every paraphrase to top-1. The full per-paraphrase rank data are tabulated in Appendix L.

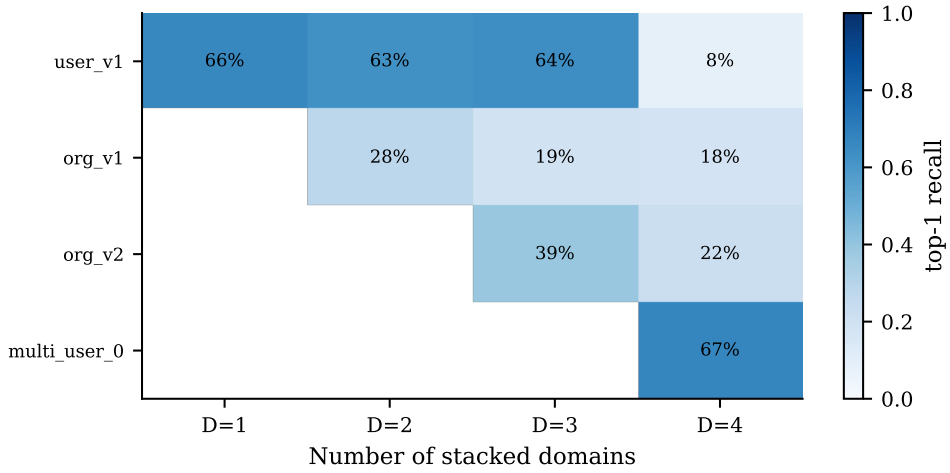


Figure 26. Multi-domain additive composition heatmap. Each cell shows per-domain top-1 recall when D domains are stacked. Heavy degradation when domains share trigger templates (high address overlap). Disjoint-template domains (corp + user, demonstrated in main text) compose without loss.

Appendix G: Multi-domain composition

Figure 26 stacks D fact domains into one Engram table and reports per-domain top-1 as D grows. The user domain (`user_v1`) is unaffected by adding disjoint-template corporate domains: it holds 66% → 63% → 64% as `org_v1` and `org_v2` are layered in (D=1 to 3). The collapse appears at D=4, when a second *user-template* domain (`multi_user_0`) is added: because it reuses the same trigger templates, the two user domains collide in the address hash and `user_v1` falls to 8% while the newcomer reads 67%. Domains that share templates interfere (the corporate domains also degrade each other, `org_v1` 28% → 18%), whereas template-disjoint domains compose without loss—consistent with the corp+user composition measured directly in Section 5.3.

Appendix H: Serving latency CDF

Figure 27 shows the per-request latency distribution under multi-tenant serving (the earlier `d12` v2 checkpoint, 339 M, 30 users × 50 facts, 600 single-token recall probes), and Figure 28 breaks the median request into its three components. Applying a user's override map and restoring the base table afterwards are both cheap and tightly bounded—median 2.2 ms, p99 2.3 ms each—so the swap adds no latency tail. The forward pass dominates: a median of 16.5 ms out of a 23.2 ms end-to-end total (p99 27.8 ms), so per-user memory adds under 20% overhead on top of a request the model would run anyway. These absolute numbers are higher than the ablation-optimal `d12@1280` serving run reported in Figure 19 (0.03 ms apply, 4.4 ms p50, 226 req/s on an idle GPU)—both the apply cost and the forward pass are faster on that newer benchmark—but the qualitative conclusion is the same in both: the apply/restore swap is a small fraction of per-request latency, and per-request work is independent of how many tenants share the server.

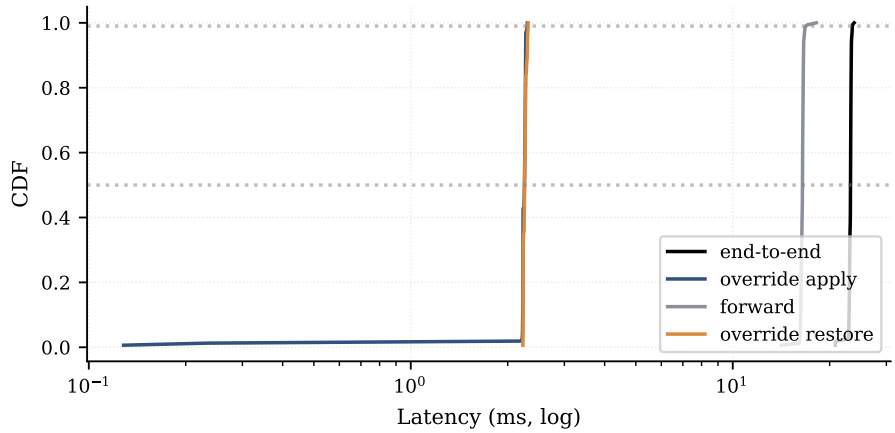


Figure 27. Per-request latency CDF on Mini-Engram-d12 (30 users × 50 facts × 600 requests). Override apply (blue) and restore (orange) are both sub-3 ms at p99; the forward pass (grey) dominates total latency (black).

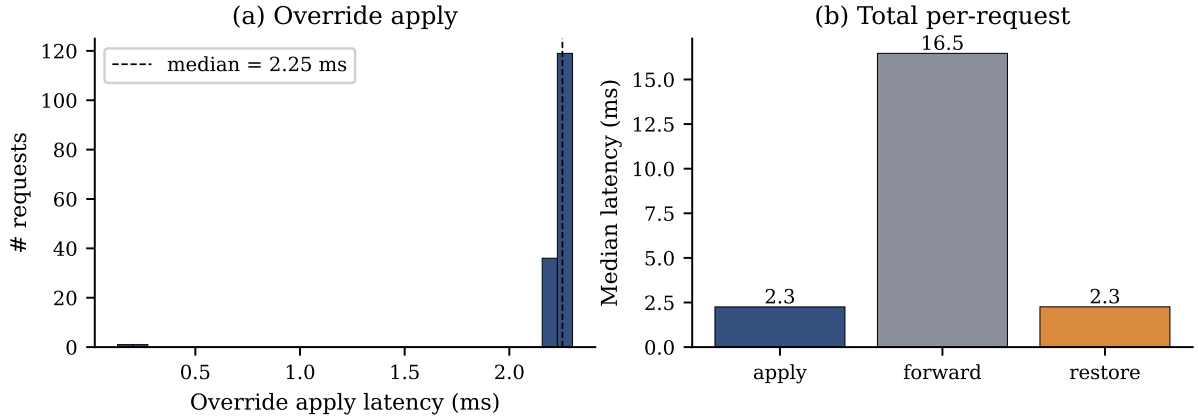


Figure 28. Per-request latency breakdown, Mini-Engram-d12 (30 users × 50 facts × 600 requests). **(a)** Override-apply latency: median 2.2 ms. **(b)** Median per-request latency by component: apply (2.3 ms) + forward (16.5 ms) + restore (2.3 ms); the frozen forward pass dominates, and the override apply+restore overhead (∼4.6 ms) is under 20% of the 23.2 ms end-to-end median.

Appendix I: Qwen-3B + RAG details

The Qwen-3B + RAG measurements in Section 6.4 use `Qwen/Qwen2.5-3B-Instruct` in bf16 with greedy decoding ($\leq$ 32 new tokens). The chat-template prompt is:

```
[SYSTEM]
You are answering personal-memory questions about the user. Use only
the facts provided. Answer with the shortest possible span (typically
1-3 words). Do not explain or add any extra text.

[USER]
Facts about the user:
- <retrieved fact 1>.
- <retrieved fact 2>.
...
```

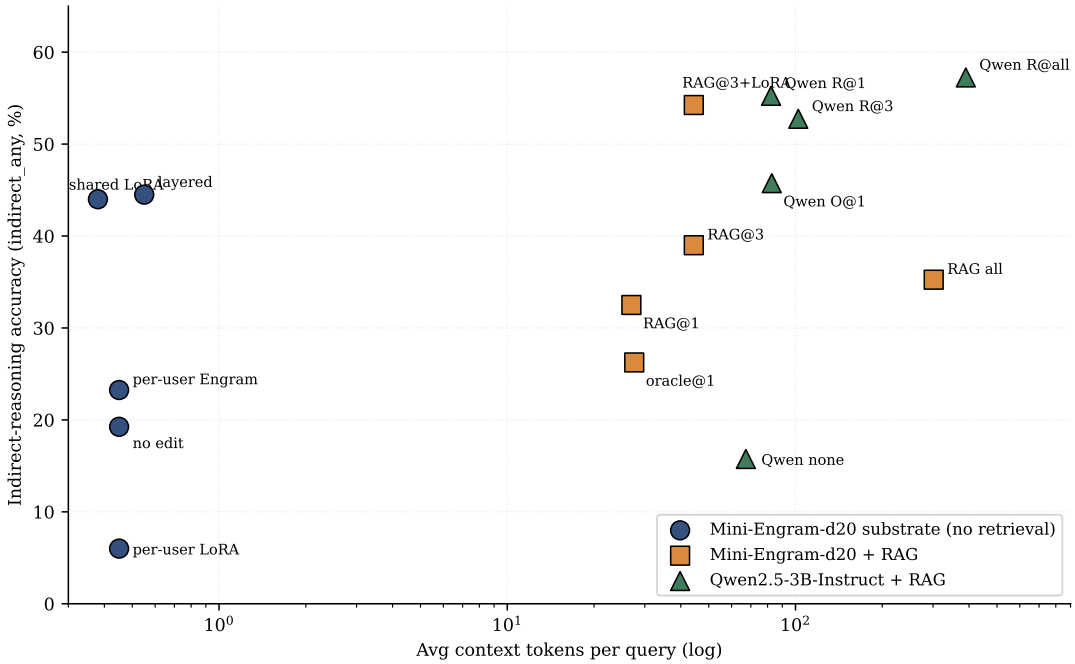


Figure 29. Indirect-reasoning accuracy (`indirect_any`) vs. avg context tokens per query on the 20-user split. The layered design and the shared LoRA alone anchor the 0-context end; RAG top-3 + shared LoRA lifts accuracy to 54% at 44 context tokens; Qwen-3B + RAG reaches 55–57% at 82–390 tokens on a $\sim$2.5$\times$ larger backbone. Naive RAG on the same Mini-Engram-d20 base sits *below* the layered design: putting facts in context is not the same as having the reasoning skill.

```
Question: <indirect question from indirect_qa>
Answer:
```

Retrieval uses the same `all-MiniLM-L6-v2` encoder as the RAG/MEM0/MEMMACHINE baselines of Section 5.4, with the per-user fact set as the index. For each indirect probe the question text is embedded and cosine-similarity top-k retrieval returns the facts to put in the context block. The same 20-user/20-probe split as Figure 17 is evaluated.

Two answer-match criteria are reported: *indirect_top1* requires the first generated word to equal (case-insensitive) the first word of the gold answer; *indirect_any* requires the gold span to occur anywhere in the generation. This matches the substring matcher used for the layered design.

Why oracle-top-1 underperforms RAG top-1. A non-obvious result in the Qwen-3B + RAG experiment is that oracle top-1 retrieval (using the known required facts to fetch the ground-truth fact) gives 46% indirect_any, *below* RAG top-1's 55%. Inspecting the data, retrieval often returns facts that are *adjacent* to the required key in semantic space and that happen to also contain the answer surface (e.g. retrieving the "birth-year" fact when asked about "current age" covers both the year and the implicit age). RAG with $k = 1$ thus over-samples the information needed for indirect reasoning, while strict oracle retrieval leaves the LM to derive the answer from one single-attribute fact—which Qwen-3B often refuses to do under our "shortest span" system prompt.

Table 8. Shared-table-with-salt: own-fact recall vs. leak rate. All numbers on Mini-Engram-d12 with the XL corpus.

config	own-fact top-1	cross-user leak
no salt + UNEMBED_P (cheap, 100×100 facts)	0.089	0.039
salt + OPT-15 (10×30 facts)	0.11	**0.000**
salt + strong OPT-60 (10×30)	0.46	**0.000**
salt + strong OPT-60 (30×100)	0.345	0.005

Appendix J: Alternative architectures we considered

For completeness we report two alternative designs we benchmarked before settling on per-user override tables.

Shared table with per-user hash salt (deprecated). In this design, every user's facts coexist in one physical table; their hashes are XOR-salted by user-id so identical surface triggers map to disjoint rows. We benchmark on Mini-Engram-d12 (Table 8):

The salt design provides architectural privacy at the cost of recall: salted addresses point to untrained rows (the model never saw salt during pretraining), so the gate fires more weakly and OPT must work harder. Because per-user override tables (main body Section 7) provide stronger privacy by design *and* use the trained address space, we recommend overrides as the production design and keep salt as a fallback for cases where the storage backend cannot afford per-user override maps.

100-user serving: UNEMBED_P vs. Joint OPT. At 100 users × 100 facts on Mini-Engram-d12@1280, the cheap (closed-form) UNEMBED_P strategy reaches 9.2% top-1 / 23.4% top-5 at 2.3 ms apply latency. Joint OPT (1000 steps, the production setting) on the same configuration reaches **62% top-1 / 96% top-5** at **226 req/s and 4.4 ms p50 latency** on an idle GPU (Figure 19), matching the ∼60%/95% prediction extrapolated from the 30-user data and consistent with the per-user-density curve at d12@1280 (Section 5.2: 67% top-1 / 99% top-5 at 100 facts/user). The serving-protocol cross-user leak under Joint OPT is 6.1% from gold-value coincidence (7 of 114 cross-user probes), versus 46% under the all-coresident protocol where every user's overrides occupy the same table at once.

Appendix K: Insertion strategies (full)

Table 9 reports the first comparison of insertion strategies on Mini-Engram-d8 with the original 16-fact USER+ORG benchmark (single-fact-at-a-time):

The DUAL strategy (jointly satisfying gate-K and value-V via least squares) was tested, but proved *worse* than UNEMBED_P (4/16 +Δlogit)—over-constraining e drives it off the trained-row distribution and the gate suppresses it. We dropped DUAL.

Appendix L: Per-paraphrase rank table

Table 10 gives the full per-paraphrase rank data behind Section 5.4 and Figure 25, single-trigger insertion on Mini-Engram-d8:

Table 9. Insertion strategy comparison on the original 8 USER + 8 ORG benchmark, Mini-Engram-d8. RANDOM is a control; UNEMBED_P is the closed-form pseudo-inverse; OPT is 15-step gradient on the row. Numbers are mean across the 16 facts.

strategy	mean Δlogit	+Δlogit	rank improved	top-1 hits	top-5 hits
RANDOM	–4.708	1/16	2/16	0/16	0/16
WTE	–0.150	6/16	6/16	0/16	1/16
UNEMBED_P	+1.711	15/16	13/16	1/16	3/16
OPT (15 steps)	**+5.323**	**16/16**	**15/16**	**6/16**	**7/16**

Table 10. Per-paraphrase rank of the gold token after single-trigger OPT insertion. "⋆" marks rank 0 (top-1).

insertion trigger	query (paraphrase)	rank
My doctor's name is Dr.	My doctor's name is Dr.	0 ⋆
	My doctor is Dr.	0 ⋆
	Who is my doctor? Dr.	54
	My physician's name is Dr.	0 ⋆
	My GP is Dr.	0 ⋆
My favorite spice is	My favorite spice is	0 ⋆
	I love the spice	1399
	My preferred spice is	32
	The spice I like most is	45
	My go-to spice is	94
Globex office hours start at	Globex office hours start at	0 ⋆
	Globex opens at	1
	Globex starts work at	0 ⋆
	What time does Globex open? At	2
	Globex's day begins at	0 ⋆
Stark Industries headquarters is in	Stark Industries headquarters is in	0 ⋆
	Stark HQ is in	1949
	Stark is based in	216
	Stark Industries is located in	66
	The Stark headquarters is in	0 ⋆

Generalization is high when paraphrases share suffix N-grams (e.g., all "doctor" paraphrases end in "Dr."); poor when the surface form diverges ("I love the spice" vs. "My favorite spice is"). Inserting at all 5 paraphrases per fact (multi-trigger) drives every paraphrase to rank 0.

Appendix M: Joint OPT convergence

Figure 30 traces the Joint OPT optimization behind the density curves of Section 5.2. At 100 facts/user the joint objective converges to ≈0.8 cross-entropy within ∼1 500 steps; at 1 000 facts/user it plateaus near 2.0, the higher floor reflecting the within-user address interference that caps recall at high density. Convergence is monotone and stable in both regimes, so the fixed step budgets we use (2 000 steps at 100 facts, 8 000 at 1 000) leave no easy gains on the table.

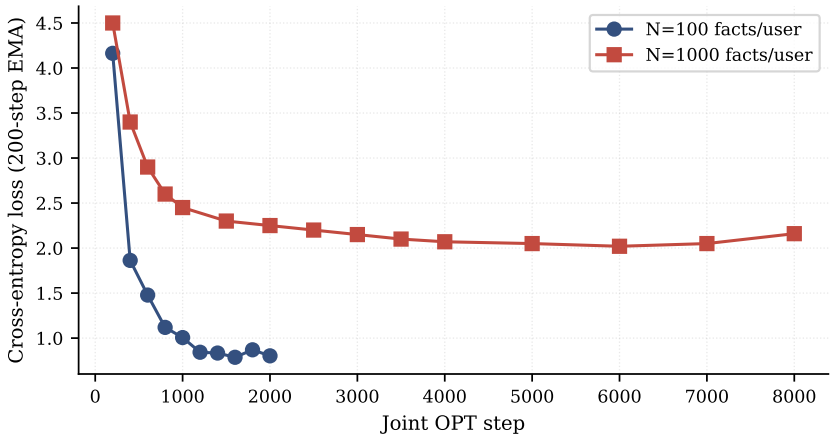


Figure 30. Joint OPT convergence on Mini-Engram-d12. At 100 facts/user, loss converges to 0.8 within 1500 steps; at 1000 facts, loss saturates around 2.0 due to higher within-user fact-density interference (Section 5.2).

Appendix N: Joint OPT algorithm

1. For each fact f: compute trigger global rows R_f (deterministic from tokens) and UNEMBED_P initial marker $e_f^{(0)} = W_V^{\dagger} U_{y_f}$.
2. Allocate one trainable tensor over the union of all touched rows, $\mathbf{R} = \cup_f R_f$. Initialize: each row $\mathbf{R}[i] = \text{mean}_{f:i \in R_f} e_f^{(0)}[i]$ (average of UNEMBED_P rows where multiple facts share an address).
3. Zero out the embedding rows in $\mathbf{R}$. Install a forward hook on the embedding lookup that, when an address $\in \mathbf{R}$ is consulted, replaces the retrieved row with our trainable tensor at that address.
4. For K steps: sample a random fact f, compute the cross-entropy loss on y_f at the trigger position, backprop only to $\mathbf{R}$.
5. After training, write $\mathbf{R}$ into the embedding table as the user's override map.

We use $K = 2000$ for 100 facts and $K = 8000$ for 1000 facts, Adam LR 0.5. Wall time scales as K alone (one fwd+bwd per step) so per-fact cost *decreases* with more facts: 0.44 s/fact at N=100, 0.23 s/fact at N=1000.

Appendix O: Seed variance of the layered measurement

The headline layered-design vs. per-user-LoRA numbers in Figure 17 are from a single LoRA training run; for transparency, we re-ran the same configuration with two further RNG states and compared (Table 11). Because the training-data ordering, LoRA-weight initialization, and Adam-state warmup are stochastic, per-user val_bpb Δ varies across seeds even with identical hyperparameters.

Reading: the architectural-property comparison (the per-user Engram's Δbpb $\ll$ per-user LoRA's by $\sim$33,000$\times$) is seed-stable to four decimal places—every seed's Engram is at the noise floor and every seed's LoRA is in $[+1.56, +1.78]$. The conclusion that the layered design beats per-user LoRA on every measure is also seed-stable: the layered design is never worse than

Table 11. Three-seed variance on the layered-design vs. per-user-LoRA contrast, Mini-Engram-d20, n=20 users per seed, identical config across seeds (rank-64 LoRA, 1500 steps, lr 5e-4, lora_alpha 128; per-user Engram J-OPT 1500 steps lr 0.5; eval_tokens 524288; only seeds differ in LoRA-weight initialization and Adam-state / data-ordering RNG). The architectural property (per-user Engram Δbpb at noise floor $\sim 10^{-5}$) is seed-invariant; the LoRA contamination magnitude varies by ∼13% and the layered design's indirect_any varies by ∼10 pp across seeds while the qualitative ranking (layered ≫ LoRA on indirect, LoRA ≫ layered on contamination) is preserved in every seed.

quantity	seed S0	seed S1	seed S2	3-seed mean (range)
per-user LoRA Δbpb mean	+1.784	+1.754	+1.557	+1.698 [+1.56, +1.78]
per-user LoRA Δbpb per-user range	[+0.44, +3.74]	[+0.73, +2.46]	[+0.69, +2.89]	—
per-user Engram Δbpb mean	+0.000052	+0.000049	+0.000054	+0.000052
LoRA+Engram stack Δbpb mean	+1.819	+1.756	+1.290	+1.622 [+1.29, +1.82]
layered design Δbpb mean	+0.386	+0.388	+0.379	+0.385 [+0.38, +0.39]
layered design indirect_any	44.5%	44.2%	34.8%	41.2% [34.8, 44.5] %
per-user LoRA indirect_any	6.0%	8.8%	7.2%	7.3% [6.0, 8.8] %
layered design direct top-1	100%	100%	100%	100%
per-user LoRA direct top-1	99%	97.5%	100%	98.8% [97.5, 100] %
layered design worse than base	0/20	0/20	0/20	**0/60**
per-user LoRA worse than base	17/20	16/20	16/20	**49/60 (82%)**
LoRA+Engram stack worse than base	18/20	16/20	16/20	**50/60 (83%)**

the no-edit base on indirect (0/60 across seeds), while per-user LoRA is worse in 49/60 = 82%. The exact magnitudes vary modestly: seed-to-seed layered indirect_any spans 34.8%–44.5% (mean 41.2%), and the layered-vs-LoRA indirect ratio spans 4.8× (S2) to 7.4× (S0). The body headlines the canonical seed S0 (7.4×, 44% vs. 6%) and cites the 3-seed mean (5.6×, 41% vs. 7%) alongside it; this appendix makes the per-seed variance explicit so readers can see which numbers depend on which seed.

Appendix P: Extended experiments and ablations

This appendix collects the capacity, fact-count, dense-size, and rank ablations, and the layered-architecture robustness studies, that the main text summarizes in figures.

P.1 Multi-fact-in-the-loss finetune, and the teacher-trace negative result

We test the multi-fact-in-the-loss recipe via a continuation finetune of d12@1280 that injects many synthetic foreign rows per batch and adapts the gate, value projection, and dense backbone to handle them, while general LM ability is preserved (val_bpb unchanged). At a fixed inference budget it improves recall at high density (+14% relative at n=1000, 8k steps); with the budget unpinned (12k steps) both the baseline and MF reach ∼34.5% at n=1000 (Figure 31). So MF accelerates Joint-OPT convergence rather than raising the asymptotic ceiling—an economic win, not an architectural one. The teacher-trace variants (245 Qwen3-8B [Qwen Team, 2025b] traces of observation/statement/QA/⟨think⟩-reasoning) cut contamination (Δbpb +0.386 → +0.233/+0.196) but lowered indirect_any (44.5% → 38.5%/28.0%; bootstrap 95% CIs strictly negative), because that trace format scores a ⟨think⟩ block while our eval reads only the top answer token.

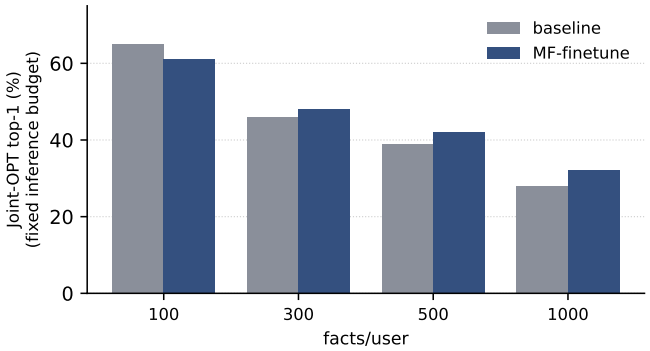


Figure 31. Multi-fact-in-the-loss finetune vs. baseline d12@1280 on the Joint-OPT density curve (matched OPT-step schedule). At a fixed inference budget MF improves recall at high density (+14% relative at n=1000, 8k steps). With the budget unpinned (12k steps) both reach ∼34.5% at n=1000: MF *accelerates convergence* rather than raising the asymptotic ceiling. val_bpb is preserved (0.774 vs. 0.770).

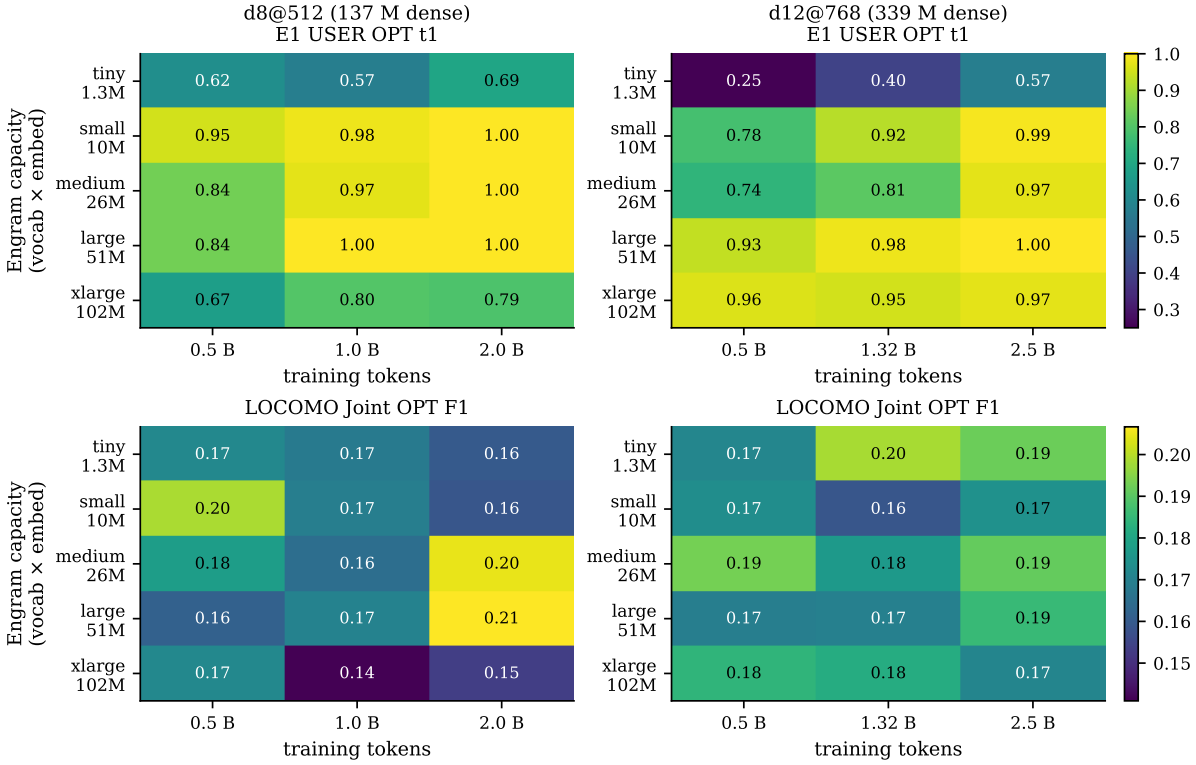


Figure 32. Engram capacity × token-budget response surface, at Mini-Engram-d8 v2 (left) and d12 v2 (right). Top row: E1 USER OPT top-1 (100-fact recall). Bottom row: LOCOMO Joint OPT F1. The same `large` (50 K × 256) Engram size wins at the highest token budget for *both* dense scales; *tiny* is under-capacity, *xlarge* is over-provisioned. The optimum scales in tokens, not capacity.

P.2 Engram capacity ablation

How does Engram-table size interact with token budget at a fixed dense backbone? We probe this as a 5 × 3 × 2 matrix: five capacity levels (v slots per N-gram, e total embed-dim per N-gram; Table 12), three token budgets, two dense sizes (Figure 32).

Table 12. Capacity levels used in the ablation. Total Engram params $= 4 \cdot v \cdot e$ (2 N-gram orders $\times$ 2 layers).

label	v slots	e embed	total Engram
tiny	5 K	64	1.28 M
small	20 K	128	10.24 M
medium	50 K	128	25.6 M
large	50 K	256	51.2 M
xlarge	100 K	256	102.4 M

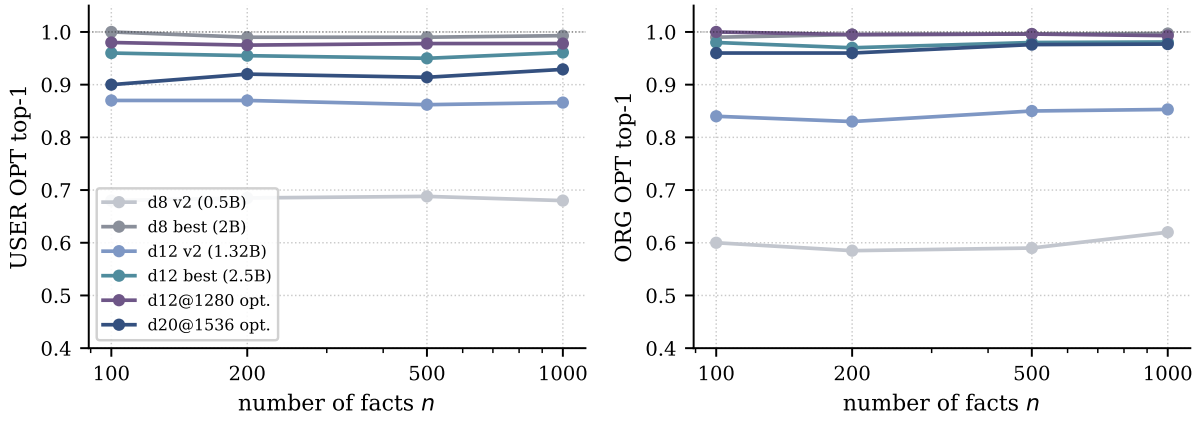


Figure 33. Per-fact independent OPT recall vs. fact count n. Left: USER OPT top-1. Right: ORG OPT top-1. The d12@1280 and d20@1536 optimal cells (green, purple) hold near-1.00 recall from $n = 100$ to $n = 1000$. No ceiling visible in this per-fact independent-OPT setting.

Three regularities. (i) *Capacity has an optimum.* Both endpoints under-perform: *tiny* cannot address enough distinct facts; *xlarge* under-trains each slot. (ii) *The optimum is the same Engram size at both dense scales*: `large` (50 K $\times$ 256 = 51.2 M params) wins at the highest token budget for both d8 and d12. **The optimum scales in tokens, not in capacity.** (iii) *Below the catch-up token budget, smaller Engrams win.* At d8 with 0.5 B tokens, *small* (10 M) beats *large* (51.2 M) on ins-OPT 1.00 vs. 0.62—the larger table is under-trained per slot. The cross-over occurs around 1 B tokens for d8 and 1.32 B tokens for d12.

Practical rule. For an Engram pretrained at Karpathy 12 t/p of scaling-params, use the `large` (50 K $\times$ 256) Engram table from d12@768 onward. For d8-class models with $\leq$ 1 B training tokens, use `small` (20 K $\times$ 128) instead. Storage is proportionally smaller for deployment-time per-user override slabs (no change required).

P.3 Fact-count scaling: per-fact independent OPT to 1000 facts

Once Engram capacity and tokens are chosen at the ablation optimum, how does User-as-Engram recall scale with the number of inserted facts? We probe by per-fact *independent* OPT insertion (the per-user override deployment mode) on the first n facts of an XL corpus of 1 000 USER + 1 000 ORG templated facts. ICL@1000 is the in-context ceiling at the hardest end. Figure 33 shows the full curves.

Per-fact recall is approximately flat in n up to 1000 facts. The best d8 and d12@1280 cells

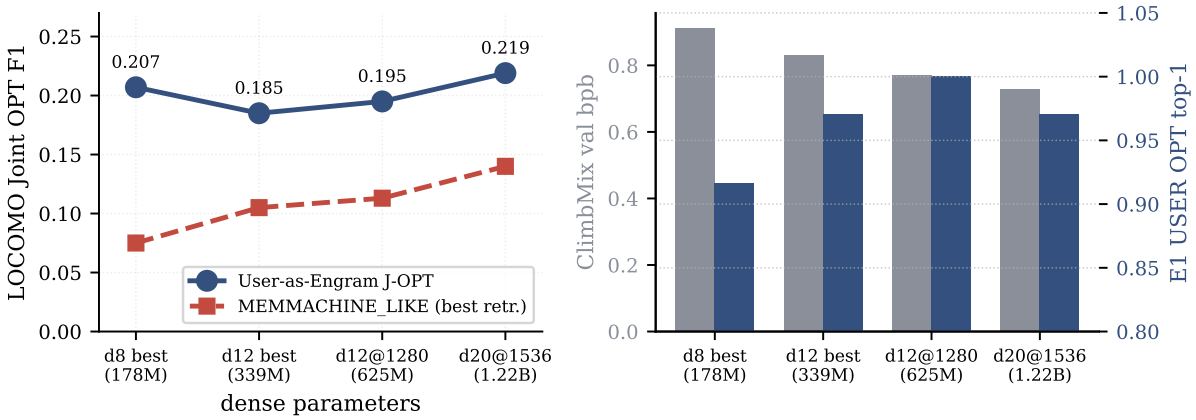


Figure 34. Dense-size scaling at the ablation-optimal recipe. Left: LOCOMO Joint OPT first-token token-F1 climbs from 0.134 (d8 v2) to 0.233 (d20@1536); MEMMACHINE_LIKE retrieval baseline climbs more slowly because its quality is bounded by sentence-encoder retrieval, not the LM. Right: val_bpb (grey) drops monotonically with dense+tokens, while E1 USER OPT single-fact recall (blue) saturates at 1.00 from d12@1280 onward (small dip at d20 at iso-12 t/p).

hold ≥ 0.98 top-1 from n=100 to n=1000. The crucial caveat is that this is *independent* per-fact OPT: each fact gets its own private hash rows, with no within-table interference between facts—the per-user override deployment mode of Section 4. *In that setting, we observe no recall ceiling up to 1,000 facts on a 625 M-parameter base LM.* The density ceiling appears under Joint OPT into a shared table (Section 5.2): 68% $\rightarrow$ 35% top-1 from 100 to 1,000 facts at d12@768.

P.4 Dense-size scaling at optimal config

Pulling the four trained Mini-Engrams into one row each (Figure 34):

Three observations. (i) **Val bpb scales smoothly with dense $\times$ tokens**: 0.91 $\rightarrow$ 0.83 $\rightarrow$ 0.77 $\rightarrow$ 0.73. (ii) **Insertion-OPT and E1 saturate** from d12@768 onward. The 16-fact ins-OPT probe reaches 1.00 at every dense size with $\geq$ 1.32 B tokens; E1 USER/ORG OPT reaches 1.00 at d12@1280@3.34 B, dipping slightly to 0.97 at d20@1536@7.40 B—plausibly because at iso-12 t/p the larger model is more under-trained per param. (iii) **LOCOMO Joint OPT first-token F1 scales monotonically with dense size** from 0.134 (178 M) to 0.233 (1.22 B), and the multi-token token-F1 variant climbs from 0.159 to 0.310. Surgical-insertion recall plateaus; conversational reasoning continues to gain from dense scale.

Reading the gap. Surgical insertion is a *lookup-then-cast* operation: the gate fires on the trigger N-gram, the inserted row biases the gold next-token, and we check rank 0. Once the base LM is fluent enough to make the gate selective ($\geq$ d12@768), recall saturates. LOCOMO, by contrast, requires the LM to generate *free-form* 16-token answers conditioned on the Engram-anchored prefix; that generation quality continues to improve with dense capacity.

P.5 LoRA rank ablation at 100 facts

For completeness we also ablate the multi-fact LoRA rank at fixed training budget (2000 steps, lr 5e-4, all $Q/K/V$ projections):

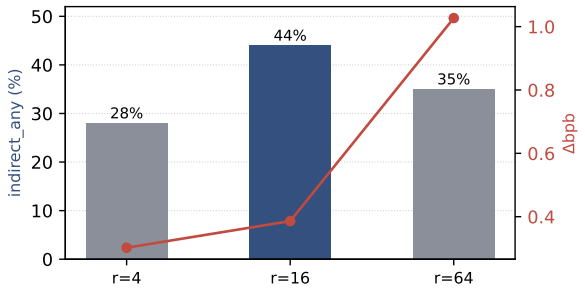


Figure 35. Shared-LoRA rank ablation (the layered design, n=20). The layered design preserves 100% direct recall at every rank; indirect_any peaks at r=16 (44%) while contamination grows with rank. The r=64 surprise: 4× the capacity gives *worse* reasoning (35%) at 2.7× the contamination, plausibly because capacity exceeds what the 510-sample shared corpus can constrain.

Every rank ≥ 8 saturates at the recall ceiling within a 2000-step budget. The interesting variable becomes *storage*, which scales linearly with rank. The takeaway: at 100 facts/user, all rank-≥ 8 LoRAs hit the recall ceiling, so the LoRA-vs-Engram trade-off collapses to recall-vs-storage. **Engram Joint OPT is 20× smaller than the smallest LoRA at the ceiling (88 KB vs. 1.8 MB rank-8), in exchange for a ∼35-pt top-1 gap (65% vs. 100%; top-5 gap is 2 pt: 98% vs. 100%).** That gap closes by n=1,000, where LoRA rank-64 also slips to 38% top-1 (Section 5.2).

P.6 Inserted-fact recall in free continuation

Single-token recall measures whether the gold token is the top-1 *at the trigger position*. In conversational use the model continues for many tokens; we test whether the inserted fact surfaces in the first 8 generated tokens (greedy decoding) on 30 facts after Joint OPT (1500 steps).

Dense scaling sharpens generation behavior. At the smallest scale the gold token is the immediate top-1 only 23% of the time but *anywhere in the next 8 tokens* 53% of the time—the model "approaches" the fact across a few tokens. From d12@1280 onward, the gold either lands at position 0 or never appears in the window: the bigger model commits earlier rather than drifting. We caution that n=30 is small (Wilson 95% CI on 27/30 is $[74\%, 97\%]$; on 18/30 is $[42\%, 75\%]$), and the result is consistent with the bigger model simply missing the fact when its first guess is wrong. Either way, the 90% first-token rate at d20@1536 is the practical recall in conversational use.

P.7 Rank ablation: r=16 is the sweet spot

We sweep the shared LoRA rank $r \in \{4, 16, 64\}$ on the full 20-user split (Figure 35) and observe a clear, non-monotonic curve. Smaller ranks under-fit the reasoning skill; larger ranks both contaminate more (more parameters, more global perturbation) *and* learn the reasoning skill worse, presumably because the LoRA's capacity exceeds what the 510-sample training corpus can profitably constrain.

The r=64 row is the surprise: a 4× larger shared LoRA gives *worse* indirect performance (35% vs. 44% at r=16) at 2.7× more contamination. We recommend r=16; future work could revisit this once the shared training corpus is scaled beyond 510 samples.

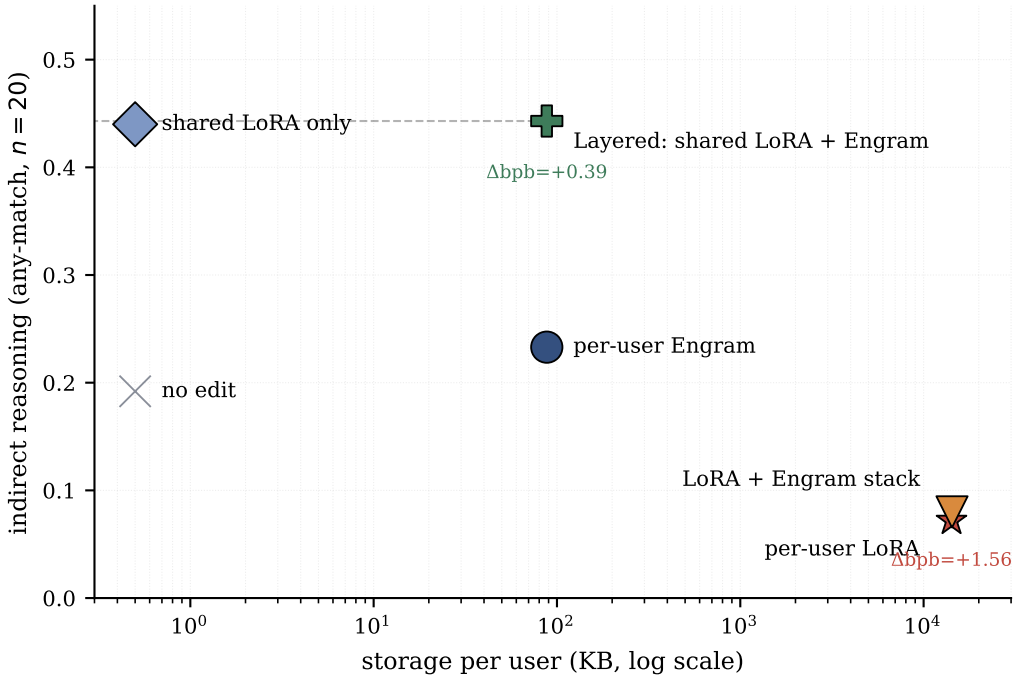


Figure 36. Cost-quality trade-off for the layered architecture (Section 6). Same Mini-Engram-d20 base, n=20 test users, six method combinations. **The layered design gives the best trade-off**: it matches the highest indirect-reasoning accuracy (44% indirect_any) at low storage (88 KB/user) and low contamination (Δbpb +0.39). It beats per-user LoRA on every measure. The naive LoRA+Engram stack (the "add Engram on top of LoRA" baseline) is beaten too.

P.8 Cross-schema generalization

The headline layered-design vs. per-user-LoRA numbers train the shared LoRA on held-out users in the same synthetic schema as the test users. We test whether the reasoning skill transfers *across* schemas. We train the rank-16 shared LoRA on the original first-person personal-facts schema (u020–u029; "My name is…", "I was born in…") and evaluate the full six-condition head-to-head on a held-out medical schema (m000–m019, third-person patient framing: "The patient's name is…", "MRN: …"; 29 facts, 20 indirect probes per user; Figure 37).

The layered claim survives cross-schema, with a realistic decay. The layered design's indirect_any degrades from 44% (within-schema, Figure 17) to 31% (cross-schema, Figure 37)—a 30% relative drop. Over the same gap, per-user LoRA's indirect_any goes from 6% (within-schema) to 4% (cross-schema), so the layered design's lead is preserved (7.4× within-schema; 7.6× cross-schema). The architectural locality is guaranteed by design: the layered design's Δbpb equals the shared LoRA's, +0.386, in both cases, identical to four decimal places, because the per-user Engram still fires only on its trigger N-grams. *Architectural decomposition transfers across schemas.*

P.9 The layered design on a Q/A-format-adapted Engram base

The headline layered-design vs. per-user-LoRA numbers are on a base LM (Mini-Engram-d20). Section 3's cross-base table shows that per-user LoRA's *user-visible* damage shrinks dramatically

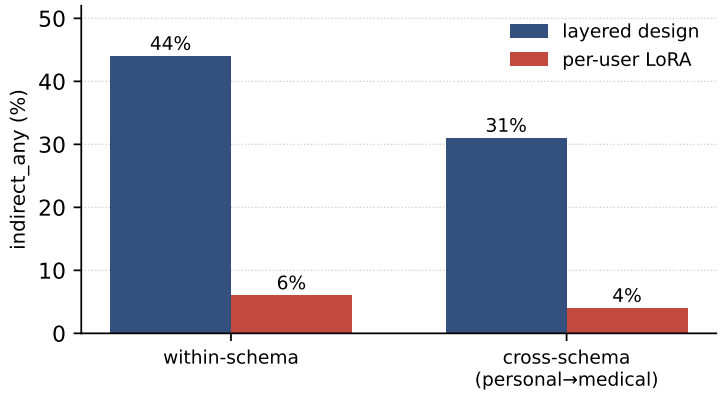


Figure 37. The layered claim survives a personal → medical schema shift (shared LoRA trained on personal-schema users, tested on medical-schema users). The layered design's indirect_any decays from 44% to 31% (a realistic 30% relative drop) while per-user LoRA goes 6% → 4%, so the layered design's lead is preserved (7.4× → 7.6×); locality is unchanged (the layered design's Δbpb equals the shared LoRA's, +0.386).

on instruction-tuned bases (85% of users worse on a base LM, 0–20% on four instruction-tuned bases). Does the layered design still dominate per-user LoRA when the latter itself becomes a stronger baseline?

We SFT Mini-Engram-d20 on ∼2000 single-token (Q, A) pairs from the same fact templates the layered experiment uses (1500 steps; lr 1×10^{-4}; post-embedding params trained; Engram tables frozen). The SFT is intentionally narrow—no general-text corpus mixed in—so it tests Q/A format adaptation rather than full chat-tuning. val_bpb on raw ClimbMix rises from 0.73 to 4.42, indicating that the model has been pushed toward the Q/A distribution at the cost of general-text LM ability. We then train a fresh rank-16 shared LoRA on this SFT'd base (`shared_lora_d20_sft/r16`, 2000 steps on u020–u029) and rerun the six-condition head-to-head; the numbers below carry the conclusion.

The layered design still beats per-user LoRA on every measure; the lead shrinks as the baseline strengthens. The qualitative picture survives Q/A adaptation: the layered design matches per-user LoRA on direct (100% vs. 100%), beats it on indirect (1.6× on top-1, 2.8× on indirect_any), and the architectural locality is preserved (the layered design and the shared LoRA alone differ in Δbpb by 1×10^{-4}, on the same order as the 5×10^{-5} original-base noise floor). The cross-base prediction reproduces: per-user LoRA's indirect_top1 jumps from 1.3% (untouched base) to 23.5% on this adapted base, matching Section 3's "instruction-tuned bases absorb the LoRA perturbation" pattern. The layered design's lead over per-user LoRA on indirect_any compresses from 7.4× (base LM) to 2.8× (SFT'd), but it remains the best method.

Caveats. The SFT here is narrow Q/A adaptation, not full chat-tuning: mixing general-text in the SFT corpus or starting from a true instruction-tuned base (the cross-base table's Qwen/Llama/Mistral setup, but with an Engram-pretrained backbone) is the natural follow-up. The aggressive SFT also collapses the untouched base's and the per-user Engram's indirect to floor (1%), which inflates the layered design's relative lead over the untouched base but not the layered-vs-LoRA contrast that matters for deciding which method to use.

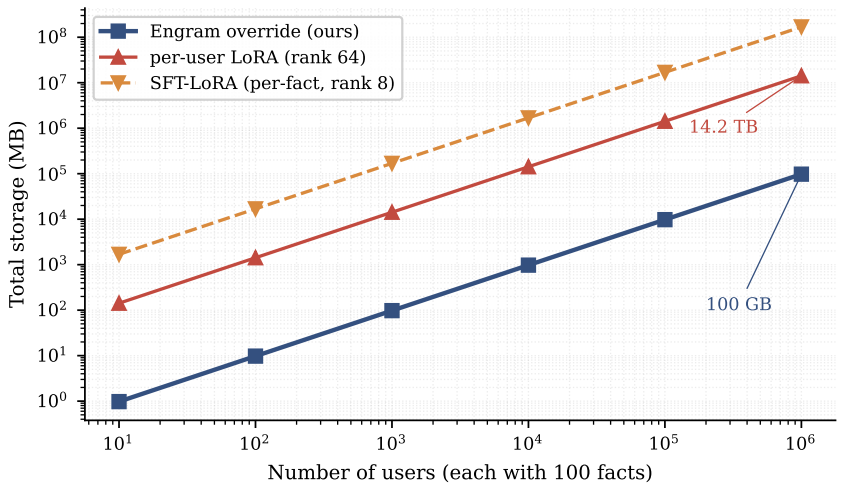


Figure 38. Per-user storage scales linearly with user count. Engram override is $\sim$161$\times$ smaller at 100 facts/user (88 KB vs. 14.2 MB) and the gap widens to $\sim$1700$\times$ at 10 facts/user (LoRA's parameter count is fact-count-independent while Engram grows linearly). At 1 M users $\times$ 100 facts/user, Engram storage is 100 GB vs. per-user LoRA's 14.2 TB.

P.10 Storage and cost sharing

Per user, the layered design costs the Engram's 88 KB; the shared LoRA is one global 11.8 MB (rank-16) file shared by all users. For 1 M users that is 100 GB + 12 MB $\approx$ 100 GB, indistinguishable from Engram alone, versus 14.2 TB for per-user LoRA at the same recall ceiling. Figure 36 plots all six method combinations against storage and indirect-reasoning accuracy.